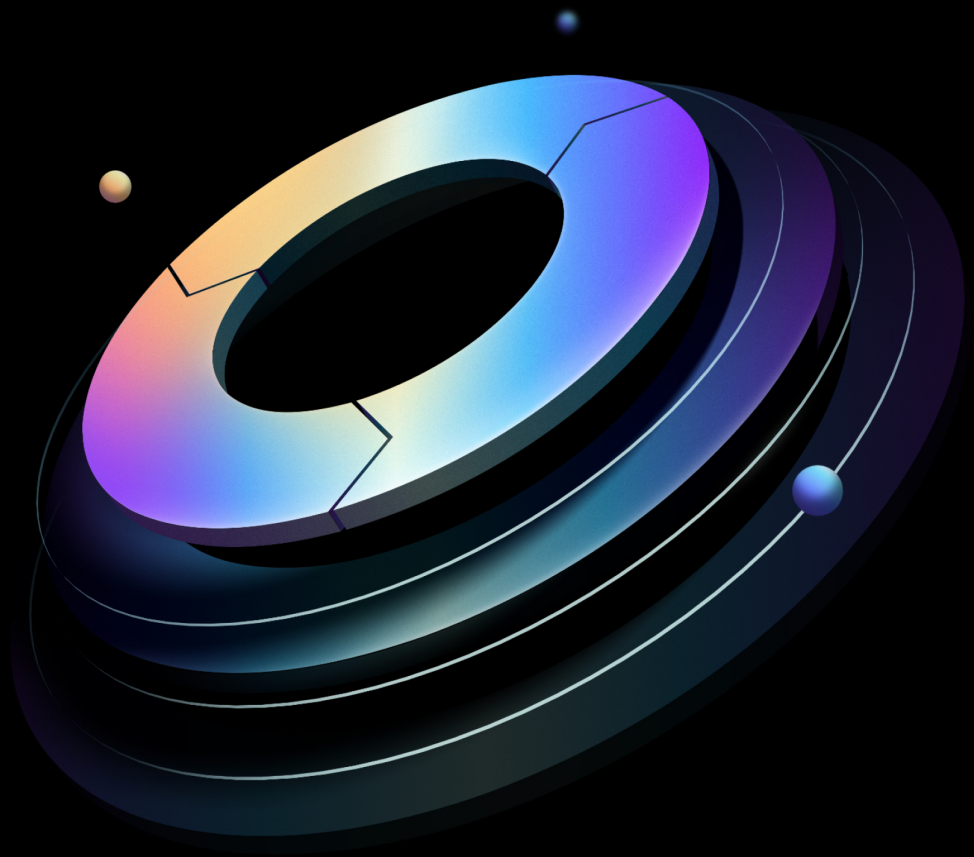
HashiCorp

Consul Administration Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **4**
1.1 Why use HashiCorp Validated Designs? . . . 4
1.2 Objectives . . . 5
1.3 Prerequisites . . . 5
1.4 Architecture assumptions . . . 6
1.5 Language and definitions . . . 8
1.6 Sample project plan . . . 9

2 People and process **11**

3 Cloud operating model **12**

4 Organizational structure **15**
4.1 Teams . . . 15
4.2 Personas . . . 16
4.3 Self service workflows . . . 16

5 Identity and access management **19**
5.1 Legacy initial configuration scope . . . 19
5.2 Configure Consul as code . . . 20
5.2.1 Terraform . . . 21
5.2.2 API or CLI . . . 23
5.3 Configure admin partition and namespaces . . . 23
5.3.1 Configure admin partitions . . . 23
5.3.2 Configure namespaces . . . 25
5.4 Security operational procedures . . . 26
5.4.1 Rotate gossip key . . . 26
5.4.2 Rotate ca certificate . . . 26
5.4.3 Certificate TTLs . . . 27

6 Observability and alert management **27**
6.1 Prerequisites . . . 28
6.2 Build your observability strategy . . . 28

6.3 Strategy . . . 29
6.3.1 Vision . . . 29
6.3.2 Use cases . . . 29
6.3.3 Objectives . . . 29
6.4 Personas . . . 32

7 Observability pillars 33
7.1 Monitor the control plane . . . 33
7.1.1 Metrics . . . 33
7.1.2 Logs . . . 35
7.1.3 Traces . . . 37
7.2 Monitor the data plane . . . 38
7.2.1 Metrics . . . 38
7.2.2 Logs . . . 41
7.2.3 Traces . . . 43

8 Additional observability elements 46
8.1 Alerts and notifications . . . 46
8.2 Dashboards . . . 48
8.3 Processes . . . 49
8.4 Integrations . . . 49
8.5 Rollout strategy . . . 50
8.6 Project plan . . . 51
8.7 Conclusion . . . 52
8.8 Platform monitoring recommendations from initial configuration . . . 53
8.8.1 Configure platform monitoring . . . 53

9 Backup and restore operations 60
9.0.1 Backup Consul . . . 60
9.0.2 Restore Consul . . . 62

10 Upgrade procedures 62
10.0.1 Upgrades . . . 63

11 Multi-cluster and multi-tenant deployments 65
11.1 Consul multi-tenant . . . 66
11.1.1 Personas . . . 66
11.2 Consul namespaces . . . 66
11.2.1 Multi-tenancy with namespaces . . . 67
11.3 Consul admin partitions . . . 68
11.3.1 Multi-tenancy with admin partitions . . . 69
11.4 Consul multi-cluster . . . 72
11.4.1 Cluster peering . . . 73
11.5 Conclusion . . . 76

12 Support readiness 76

13 Changelog 76
13.1 2026-08 . . . 77
13.2 2026-01 . . . 77
13.3 2025-06 . . . 77
13.4 2025-05 . . . 77
13.5 2025-01 . . . 77
13.6 2024-08 . . . 77
13.7 2024-02 . . . 77

14 Credits 78

1 Introduction

Note

These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

This Administration Guide is for platform and operations teams responsible for running Consul Enterprise after installation. It assumes the Consul Installation Guide has been completed and the system is running. For application-team service discovery and service mesh workflows, see the Consul User Guide.

Note

The current version of this guide contains recommendations for operating Consul Enterprise on AWS EC2 with client workload deployments on AWS EKS. Future versions will include deployment options for other cloud providers, including Azure, GCP, and on-premises deployments. They will also include information for running clients in additional Kubernetes platform varieties.

1.2 Objectives

This guide covers operating Consul Enterprise as a platform service. Consul lets you register services in a centralized registry to discover, monitor, and track their health status. It also enables secure network connectivity between services across multiple cloud environments, runtimes, and platforms.

Deploying Consul will help your organization provide the following solutions:

1. **Identity-based networking** provides a dynamic way of referencing services by their identity. This lets you quickly respond to scaling events without updating IP addresses for new instances inside your firewalls and load balancers.
2. **A multi-platform service discovery solution** provides a better way for applications to discover each other across different deployment platforms.

By leveraging these solutions, your organization can:

1. **Reduce the time required to scale infrastructure** by providing dynamic capabilities to discover services and get the health status to communicate securely with dynamically scaled infrastructure.
2. **Reduce the time required to release new features to consumers** Consul allows development to test new features and route traffic to old and new features based on traffic resolving and splitting rules.

> Note
>
> The recommendations in this Administration Guide are derived from different production Consul deployments. Before implementing these recommendations, carefully evaluate them and determine whether they suit your specific environment.

1.3 Prerequisites

When using Consul Enterprise (Self-managed / Self-Hosted version of HCP Consul), this guide assumes that you have a basic understanding of Consul and have reviewed or implemented the Consul Enterprise Installation Guide.

Please note: In this document, Consul Enterprise refers to both the self-managed and the HashiCorp-managed (HCP Consul) versions of Consul.

1.4 Architecture assumptions

Many of the recommendations and best practices outlined in this document are platform-agnostic and include information that could apply to any platform. When we need to highlight platform-specific details, this document focuses on specifics for running Consul Enterprise server agents deployed on Amazon EC2 instances with client workloads running in either Amazon EC2 instances or Amazon's managed Elastic Kubernetes Service within a single Amazon Region.

The following is a sample high-level architecture diagram that shows the Consul Enterprise deployment this Administration Guide recommends. The Consul Enterprise server agents are deployed by the Terraform module from the Installation Guide. Client workloads running in Amazon EKS are segregated into Consul Enterprise Admin Partitions corresponding to each Kubernetes cluster.

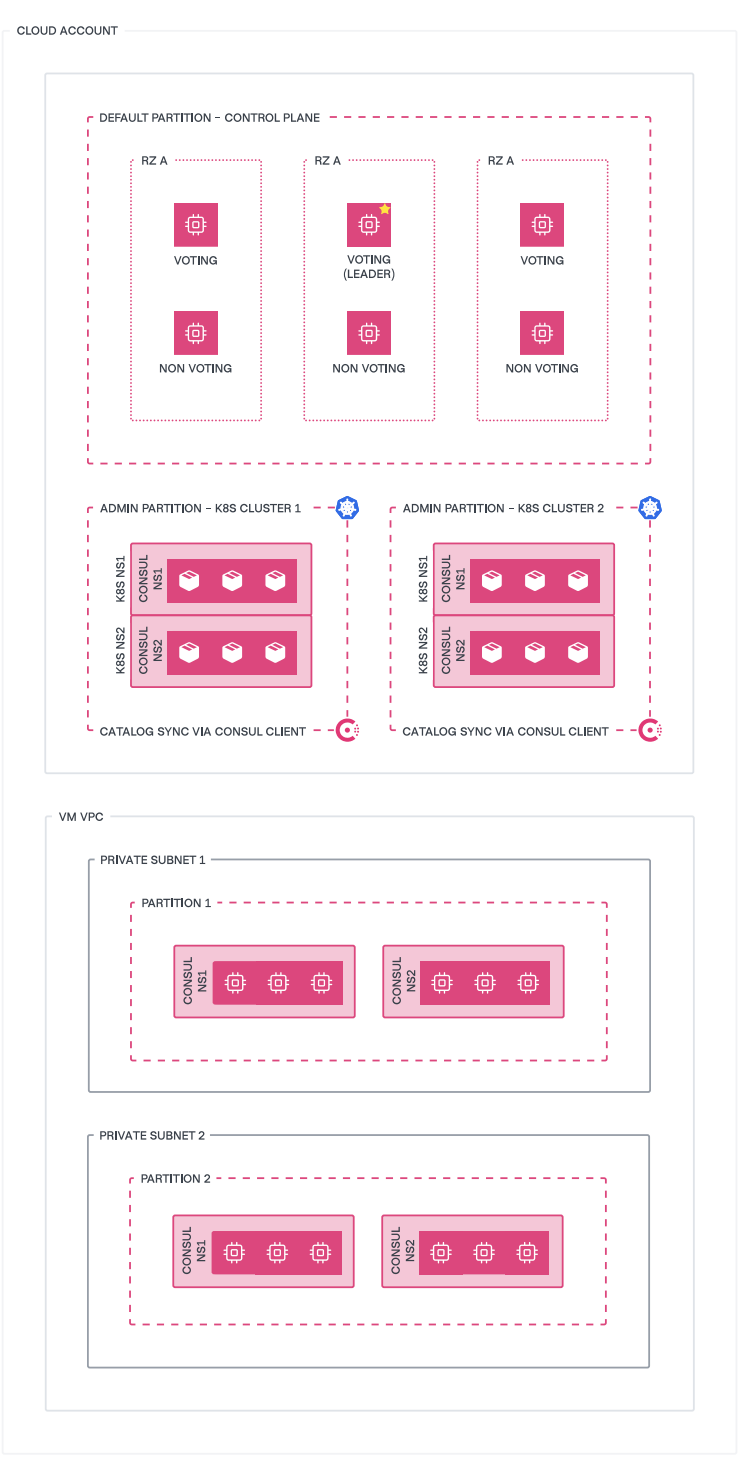


Figure 1: Consul AWS Architecture

1.5 Language and definitions

This documentation intentionally uses technology-agnostic terminology. However, some terms do not translate perfectly between the cloud providers. The following are the definitions of terms this document uses.

Cloud Provider Terms	Definition
Region	A physical location around the world with multiple clusters of data centers.
Availability zone (AZ)	One or more discrete data centers within a region. Each AZ has redundant power, networking, and connectivity.
Public subnet	A network accessible by application users.
Private subnet	A network used by applications, but inaccessible by application users.
Secrets Manager (SM)	System that can store secrets for bootstrapping.
Virtual private cloud (VPC)	Software defined cloud networking

Consul Term	Definition
Consul Enterprise	Self-managed / Self-Hosted version of HCP Consul
Datacenter	A Consul datacenter is the smallest unit of Consul infrastructure that can perform basic Consul operations. This may coincide with cloud regions' boundaries.
Partition or Admin Partition	A logical boundary within a single Consul datacenter that delineates unique network boundaries or teams.

Consul Term	Definition
Peering or Cluster Peering	A Consul datacenter or admin partition that has an established relationship with another datacenter or admin partition

1.6 Sample project plan

The following table itemizes this guide's artifacts and its expected timeline to help project managers create their project plans. You can exclude some of these activities. Reach out to your HashiCorp account team for additional guidance. They can help you customize this plan to suit your specific and unique requirements.

HashiCorp Validated Design (HVD) deliverables

Deliverable	Brief Description	Approx. time to complete
People & process		
Define user data, roles, and authentication	Define how users interact with the self-service platform and configure authentication methods, roles, and policies to be applied. Create policies as code and document team organization	1-2 weeks
Identify self-service workflow	Identify all personas and teams that will be part of the platform's consumers and producers. Document the workflow and specify inputs and outputs required to integrate with all development processes	2-3 weeks

Deliverable	Brief Description	Approx. time to complete
Initial configuration		
Configure Consul as code	Automate the initial configuration of your security policies, admin partitions and namespaces, and consumer workflows.	2 weeks
Configure DNS forwarding	Setup DNS forwarding for local agents and remote systems to allow for consumer discovery workflows.	1-2 weeks
Configure platform monitoring	Configure exporting of telemetry and logging data and integrate with your monitoring platform.	2 weeks
Setup backup, restore, and upgrades	Familiarize the team responsible for operating Consul Enterprise with the different operational procedures required to ensure the ability to adopt new features and provide resilience in the event of unexpected issues. Test and validate the procedures.	2 weeks
Security operational procedures	Validate operational rotation procedures and ensure they align with your organizational security requirements.	1-2 weeks
Multiplatform service registration and health checking		

Deliverable	Brief Description	Approx. time to complete
Build health checking workflow	Determine appropriate health checks for your applications to ensure you only direct discovery traffic to healthy instances.	2-6 weeks
Build service registration workflow	Targeting the first pilot consumers, build a service registration workflow that includes your health check definition	2-6 weeks
Service catalog discovery		
Build Consumer Workflow	Identify all of the consumers for the initially targeted applications and ensure you can perform a smooth migration from the current way they are addressing services to leveraging the Consul Service Catalog.	2-6 weeks

2 People and process

HashiCorp Consul is a service networking solution that enables teams to manage secure network connectivity between services and across on-prem and multi-cloud environments and runtimes. Consul offers service discovery, service mesh, traffic management, and automated updates to network infrastructure devices. Integrating these use cases into a common framework often requires different personas to build, maintain and use them.

Operating a networking platform, like Consul for service discovery, requires coordination between different teams to:

- Setup and maintain the platform. This may include the following:
  - Manage access to the platform and defining policies and roles
  - Monitor the platform and the services
  - Manage infrastructure and platform administration
  - Automate (or script) these tasks with the platform API (Consul API)
- Use the platform to register services. This may include the following:
  - Register application services
  - Design and implement health checks for those services

This section explains the Cloud Operating Model (COM), a validated framework that enables teams to operate and succeed in the Cloud. In particular, it will describe the teams and personas responsible for driving Consul adoption and how they interact with each other.

3 Cloud operating model

The cloud demands a new operating model. Datacenter primitives are changing: static to dynamic infrastructure, security and networking policies based on identity rather than IP addresses, and an automated self-service platform as opposed to manual ticketing systems. While this transformation speeds development and shrinks go-to-market times, it renders much of the previous generation of tools and processes obsolete. To fully realize the benefits of cloud computing, enterprises must transition to scalable dynamic workflows and management at every cloud layer. From this they can deploy shared services enabled by platform teams to improve speed, increase efficiency, and decrease risk.

The Cloud Operating Model (COM) is a new approach for IT operations that organizations need to use to be successful with cloud adoption and thrive in an era of multi-cloud architectures. This overview breaks down the components of that approach and the path to industrializing application delivery across all layers needed to support a cloud-based architecture, articulating the needed changes to people, processes, and tools.

The COM uses a shared service model to organize and deliver technology services, resources, or capabilities to different parts of an organization. This approach allows different teams or business units within the organization to leverage the same set of services, reducing redundancy and promoting consistency.

There are two key roles in the COM:

1. **Producers** ensure that the shared services are readily accessible and efficiently utilized by the Consumers, empowering them to leverage the benefits of the Consul platform effectively. This is usually the platform team. Some of their responsibilities include:

- Providing a seamless onboarding process to the network automation platform, which is Consul
- Supporting the security access to Consul by configuring the right authentication method integrations and standard default policy implementations
- Providing a multi-tenant and multi-region framework to discover and connect services by peering different Consul Datacenters and/or Partitions
- Offering on-demand processes to deploy self-service Partitions and their services
- Supporting and maintaining the documents and process to integrate with external development tools
- Offering necessary enablement to the consumers.

2. **Consumers** play a crucial role in leveraging the shared services to meet their specific needs and requirements while adhering to the guidelines and standards set by the Producers. Consumers refer to any team within the organization that uses the shared services provided by the platform team (Producers). This is usually the application development (AppDev) team. Some of their responsibilities include:

- Requesting access to deploy new custom policies for their tenants (or Consul Admin Partition)
- Deploying and configuring services and the health checking process
- Monitoring services health and data provided by the Consul service discovery platform
- Implementing services queries by using Consul DNS.

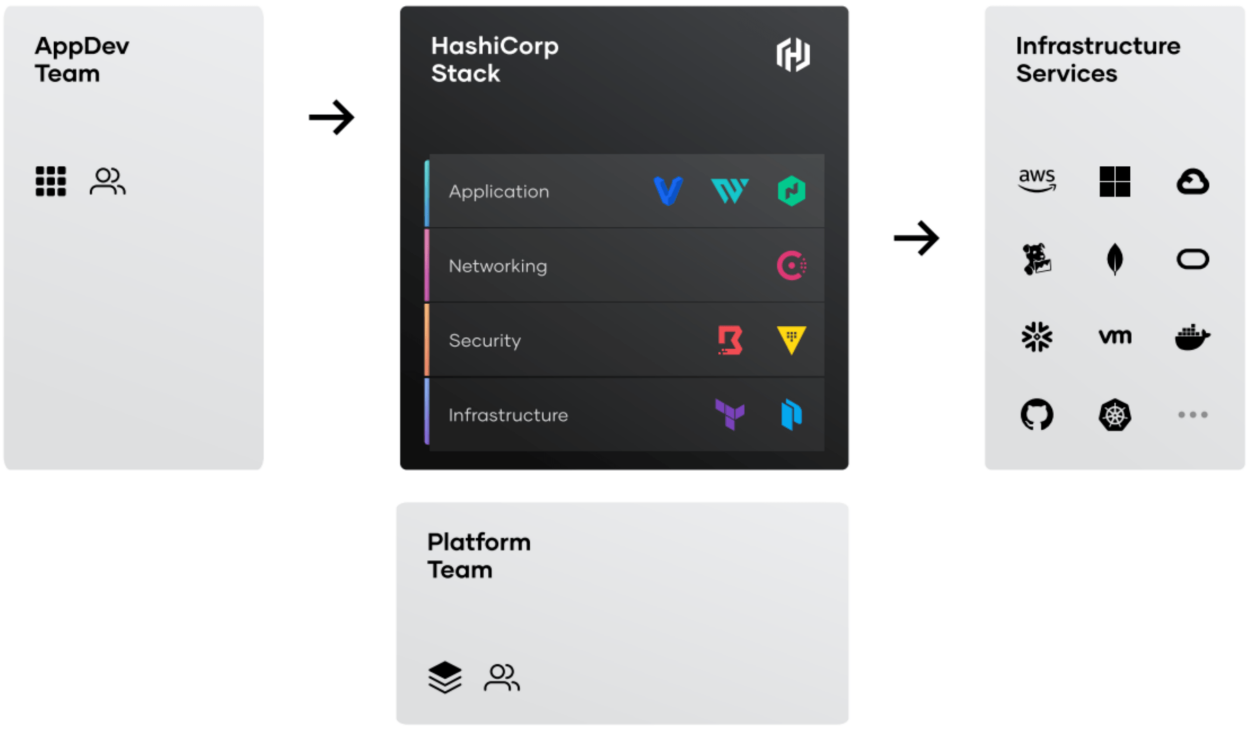


Figure 2: Cloud Adoption Framework - Teams

The platform team is responsible for delivering each layer of a cloud operating model as a standardized shared service that can be consumed by end-users in the organizations.

In addition to providing standardized shared services, the platform team:

- Establishes best practices for developers and practitioners to engage with cloud services. These best practices are then codified directly into workflows and educational programs such as Cloud Centers of Excellence (CCoEs).
- Serves as a single point of integration for other corporate teams, including security, compliance, financial controls, etc. to ensure that their best practices are baked into relevant workflows.
- Provides a central system for tracking, reporting, and auditing to inform adoption and usage of the cloud operating model. This informs progress of the organization and highlights areas where there may be risk exposure: such as security, compliance, and spending.

4 Organizational structure

Each organization has its own team and role management workflow, depending on their development lifecycle and business alignment. This section provides example teams and personas that fulfill the Cloud Operating Model's Producers and Consumers roles.

Organizations may call the team and personas described in this section different names. Focus on the infrastructure, development, networking or security activities of each team and personas rather than their specific names.

In addition, this section includes team and personas that are not responsible for operating Consul. However, they are part of the self-service workflows described in the next section.

4.1 Teams

The following teams are part of the shared service model:

- **Platform team**: Platform engineers belong to this team. They operate the services networking platform and ensure that consumers can configure and deploy their required artifacts. It can be divided into different "subteams":
  - **Service discovery specialists**: There may be experts who focus specifically on implementing and maintaining service discovery mechanisms
  - **Traffic management experts**: They are not part of the service discovery tasks, but they will need service discovery outcomes for traffic management tasks (load balancing, routing, etc.)
- **DevOps team**: Usually a cross-functional team responsible for the CI/CD framework and its integration with the discovery catalog and dns routing. Sometimes they could be part of the Platform Team, represented by DevOps engineers
- **Networking team**: Common task is to design and support the network architecture, working closely with Service Discovery Specialists
- **Security team**: They implement security measures for service communication and they ensure that any security feature or access is aligned with the organization's security policies

4.2 Personas

Within these teams, the following personas are part of the shared service model:

- **Developers** write and maintain code for the service. They may also register services to Consul, so the services can discover and connect to other services in Consul.
- **Platform engineers** are cross-discipline and can be represented by different subroles:
    - **Infrastructure specialists** are responsible for deploying and maintaining the Consul control plane components and provisioning the underlying service discovery platform.
    - **DevOps engineers** are responsible for the CI/CD setup and automating the Consul deployment.
    - **Security architects** define and implement the security framework for Consul.
    - **SREs** are responsible for monitoring and troubleshooting issues related to services health and the platform.
- **Security engineers** are responsible for implementing security policies and managing access controls, related to service registrations made by developers (e.g. they can define the access policies for different tenants, that can be namespaces or admin partitions, when registering services).
- **Network operators** ensure that the network is up and running and monitor services health provided by Consul service discovery platform to detect any network related issue. They also could be in charge of reviewing service registration for developers (e.g. ports and traffic are compliant with networking definitions) or DNS integrations.
- **Cloud engineers (infrastructure engineers)** manage automation deployment to deploy application and networking infrastructure, like load balancers, VPCs, firewalls or any infrastructure required to run applications.

4.3 Self service workflows

A self service workflow for operating and using Consul as a service discovery platform requires different steps and stages that enable teams to register and manage their services within Consul. Producers (often platform teams) provide the required platform, enabling downstream consumers (often AppDev teams) to autonomously register and manage the services that will be part of a centralized service catalog.

During the deployment phase of the application lifecycle it is important that services registration is integrated with the development lifecycle, providing a seamless process to manage a service catalog and health checking platform for the services.

The self service workflow should include the following steps, tasks or configurations:

- **User accessibility**: As a common practice, a self service portal would provide identity or token access methods with the required credentials. These will be controlled by the Consul ACL and auth methods to authorize services access
- **Service registration**: Any consumer of the Consul platform should be able to request a registration task by providing the required information, like service name, port, tags and health checking settings. The task will follow a configuration as code approach, where the service registration information is part of the development process, making it easier to integrate with any development or orchestration platform
- **Validation and triggered events**: Consumers will be able to create different scripts or events as part of the service monitoring and health checking process. Consul watches can be configured as part of required dynamic configuration for the applications deployed
- **Monitoring and alerting**: By using Consul metrics and audit logs, producers will be able to locate potential infrastructure, process or security issues. Consumers can be notified on any specific change or non-compliant updates. These metrics should be integrated with dashboarding tools that help both Producers and Consumers of the platform to identify any change or issue that could impact the application services deployment
- **Service catalog retrieval**: Consumers, like developers or network operators, will retrieve Consul services' registered information to integrate with development workflows and other networking operations like network devices configuration integration

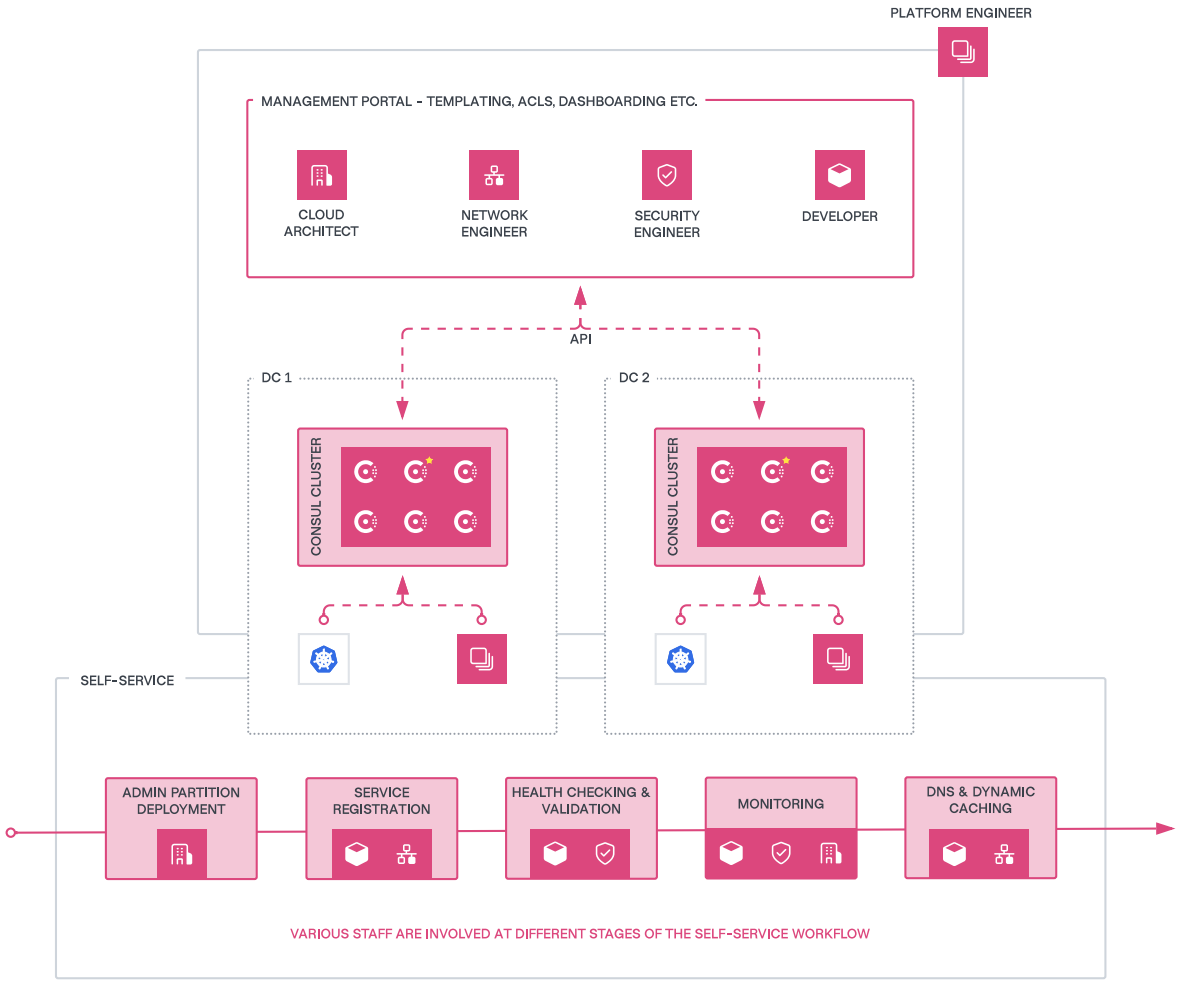


Figure 3: COF Dev Teams

From a self-service platform perspective, different roles participate in the workflow:

1. The platform team is responsible for maintaining the shared Consul control plane, the peering settings, and exported services configuration that are required for cross Consul data center workloads.
2. Infrastructure operators and architects deploy and configure the isolated Consul admin partitions depending on the specific team or business unit.
3. Developers, network engineers, or operators will register the service.

- Developers register services to Consul when they deploy them, using a specific automated process or platform (e.g. Kubernetes integration, Nomad jobs or configuration as code service definition). In addition, they should configure health checks when registering the service.
- Network engineers will use the services registration information as Consumers to configure load balancers and enable communication within the organization.
- Security engineers or operators will establish the required security compliance for the health of services.

4. Monitoring services, logs and metrics is a critical task for developers, infrastructure operators and security engineers. This will help them manage and ensure the required availability of services and infrastructure resources, and give some helpful insights about access control (e.g. blocked ACLs).
5. With a centralized service catalog, network operators can use Consul DNS and service discovery to configure load balancers or firewall rules. Developers can use Consul DNS to discover and connect services together in their environments.

5 Identity and access management

> **Note**
>
> This page covers operator management of Consul configuration, ACLs, namespaces, admin partitions, and security lifecycle tasks. For service discovery usage inside namespaces, partitions, and peered clusters, see the User Guide: Scale services on multiple infrastructures.

5.1 Legacy initial configuration scope

This section describes how the product will be configured after installation for self-managed Consul Enterprise deployments. It may take several weeks to work through all of the recommendations from each section, but this should be the primary focus of the Consul Enterprise platform operators after the initial deployment of Consul Enterprise has been completed. Establishing these foundational configuration best practices is essential before you begin targeting consumer workflows to onboard. This will ensure solid footing for your current and future use cases.

- Configure Consul as code

- Configure initial ACL roles and policies
- Configure DNS forwarding
    - Local agent resolution
    - External resolution via DNS forwarding
    - Kubernetes DNS forwarding with CoreDNS
    - Forwarding destination for external resolution and Kubernetes configurations
    - DNS ACL permissions
    - DNS performance considerations
- Configure admin partition and namespaces
    - Admin partition
    - Namespaces
- Configure platform monitoring
    - Consul control plane monitoring
    - Raft monitoring
    - Consul data plane monitoring
    - Consul platform monitoring and alerts
- Setup backup, restore, and upgrades
    - Backup Consul
    - Restore Consul
    - Upgrade Consul
- Security operational procedures
    - Gossip key rotation
    - Certificate rotation

5.2 Configure Consul as code

When managing Consul Enterprise configuration entries or other configuration resources, make automation a foundational component of your approach from the beginning. There are several correct ways to manage your Consul configuration, and different organizations adopt different GitOps and configuration management practices. However, you should avoid making manual changes to

your production Consul Enterprise environment to minimize edge cases and control the promotion of changes across your environments. Also, try not to mix different configuration styles whenever possible.

When managing Consul configuration as code, a Git repository serves as the sole mechanism for contributors and operators to modify Consul configuration through a pull request driven workflow. To establish a self service configuration workflow, you should block direct commits to the main branch linked to the production configuration and require approvals and code reviews prior to accepting pull requests. When using a code driven configuration workflow, it is important to restrict manual operator level configuration changes to reduce drift between the configuration defined as code and the state of the real world.

There are two approaches to configure service discovery within Consul Enterprise. Both approaches support all server platforms.

1. **Terraform** If you manage configuration with Terraform, it assumes control over the state of the configuration values and will overwrite any changes made when using a different configuration method. Standardizing on one approach simplifies tasks for your operators. Having a single central place to manage configuration makes it easier for operators to create new configurations or adjust existing ones. You can also leverage Sentinel policies to govern behaviors around any resources managed with the Consul Terraform provider. Terraform does not have coverage for every configuration option within Consul, but should be the primary choice when automating configuration resources.
2. **API and cli** The API has coverage for every configuration type. However, It is more challenging to build configuration workflows using the API because you need to incorporate logic around any dependencies, creation ordering, and the existing state of your configuration resources.

5.2.1 Terraform

We recommend standardizing on Terraform for maintaining your Consul Enterprise configuration resources. Some key advantages to leveraging Terraform are that it manages state across lifecycle changes, so it can track which resources have already been created and any dependencies between the resources between changes. Another advantage is that common patterns can be composed into repeatable modules so that you can collect commonly used configuration entries or security policy and easily replicate it when onboarding new teams into Consul Enterprise. Changing only the team specific inputs when referencing a common module allows you to create a consistent

usage model across similar consumer varieties.

Governance From a governance standpoint, leveraging HCP Terraform allows you to define policy around any of the Consul configuration resources using the Sentinel policy language which allows you to provide autonomy to your consumers with a self-service workflow while incorporating guardrails around any policies you want to enforce. An example of this might be enforcing the ideal quantity of servers to prevent operators from inadvertently scaling up the cluster size without understanding the quorum implications. Another example could be enforcing the inclusion of a TTL greater than zero inside prepared query resources to ensure that prepared query service queries can be cached within your DNS infrastructure.

GitOps repo structure In the earlier stages of adoption, it is common for the central team operating Consul to use one monolithic configuration repo where the policies and roles, and configuration values are defined for all consumers and Partitions while allowing for collaboration and pull requests. As you continue to mature in your usage and adoption of Consul Enterprise, you might choose to split some of the Partition specific configuration entries into their own repositories and delegate more control to the consumer teams to define configuration at their level. An example of this might be pushing service export configuration to the team responsible for managing a specific Partition where services are exported from.

An example Git configuration monorepo hierarchy:

- Consul Datacenter
  - Git Repo 1
    - Default Partition - VM Server Deployment
      - Config Entries
      - Prepared Queries
      - Auth Binding Rules
      - Cross Partition Policies and Roles
        - Cluster Wide Operators
        - Cross Partition Admins
        - Cross Partition Read Only
        - Cluster Wide GitOps/TFC Write/Management Token
      - Partition Specific Policies and Roles - TF Sub-Module
        - Partition Z Admins
        - Partition Z Read Only

- Partition Z..A - Kubernetes Cluster/VM App Team
  - Config Entries

5.2.2 API or CLI

If your organization hasn't adopted Terraform for managing application configuration, you can use the API or command line interface to automate Consul Enterprise configuration through various configuration management tools or GitOps pipeline driven automation workflows. The API or CLI offers great flexibility regarding the supported tooling, but orchestrating it might be more challenging since you'll need to track the state of configuration changes and dependencies within your chosen workflow, which can require additional code. However, one key advantage to using the API is its full coverage for all configuration types.

5.3 Configure admin partition and namespaces

Based on the architecture defined in the Installation Guide it's important to understand the purpose of admin partitions and namespaces for segmenting consumer workloads and Consul gossip communications. As part of establishing your initial consumer workloads within Kubernetes and virtual machine instances, admin partitions will be used to create a logical separation between each networking boundary.

5.3.1 Configure admin partitions

Admin partitions exist at a level above Namespaces in the Consul hierarchy. They contain one or more namespaces and allow multiple independent tenants to share a Consul datacenter.

You can use ACL tokens and permissions to manage admin partitions.

The following recommendations show you how to configure admin partitions for Kubernetes and VMs.

Kubernetes

The recommended architecture is to have each Kubernetes cluster operate as its own admin partition. As an example, if you have 10 Kubernetes clusters, you should have 10 distinct admin partitions, with each Kubernetes cluster as a unique admin partition.

You can define the admin partition in the Helm chart. The following configuration creates an admin partition named `my-admin-partition`.

```
global:
    adminPartitions:
        enabled: true
        name: "my-admin-partition"
```

Helm will create the admin partition in the non-default partition Kubernetes clusters during the installation process. Unlike with VM based environments, you do not need to manually create the admin partition first.

VM

Determining boundaries for admin partitions is more nuanced when working in non-Kubernetes environments. One of the key considerations, among others, is how gossip pools are used in Consul.

Each admin partition operates within its own unique gossip pool. Consul servers must belong to the default partition and participate in the gossip pools of every partition in the Consul datacenter.

A single gossip pool should not exceed 5000 Consul nodes. Every node within a single pool must be able to communicate over TCP and UDP with every other node in the same pool. If applications need to be network isolated from each other, you must place them within their own admin partition. This will place the services in a unique gossip pool.

You must create the admin partition using Terraform or the API before you can use them. You can define the admin partition in the Consul agent configuration:

```
partition = "my-admin-partition"
```

Service discovery with admin partitions

After you have created and deployed services to admin partitions, you must specify the admin partition in your DNS query to resolve services within a non-default admin partition. The following is the structure of the DNS query for admin partitions. Substitute service_name, namespace, and partition as applicable.

```
<service_name>.service.<namespace>.ns.<partition>.ap.consul
```

5.3.2 Configure namespaces

Consul namespaces provide a way to isolate and group resources within an admin partition within a Consul cluster. All resources are placed within a **`default`** namespace unless explicitly assigned to a non-default namespace.

One of the primary benefits of grouping resources into namespaces is to reduce service name collisions. For example, multiple applications can have a service named "database" and each will be resolved correctly within their respective namespaces.

ACL tokens and permissions are used to manage Namespaces.

Kubernetes

Consul Catalog Sync automatically syncs Kubernetes namespaces to Consul namespaces. Example, if you have a service in the xyz Kubernetes namespace, Consul catalog sync will automatically create a corresponding service in the xyz Consul namespace.

VMs

Before you can assign services to a namespace, you must first create the namespace with Terraform or the API.

Once you have a namespace, specify the namespace in the service configuration file. The following example creates a service named `my-service` in the `my-namespace` namespace.

```
service {
    name = "my-service"
    partition = "my-partition"
    namespace = "my-namespace"
    ...}
```

Service discovery with namespaces

To resolve services within a non-default Consul Namespace, the DNS query must explicitly include the namespace name:

```
<service_name>.service.<namespace>.ns.consul
```

Namespace policy defaults

If you want to allow services to discover other services located within different namespaces inside a partition, each namespace should have a policy defaults attached to it. The following is an example policy default that enables this behavior.

```
partition "default" {
    namespace_prefix "" {
        service_prefix "" {
            policy = "read"
    }
        node_prefix "" {
            policy = "read"
        }
    }
}
```

5.4 Security operational procedures

It is important to review the below security recommendations to ensure they align with your organization's security policies and standards. We recommend performing certificate and encryption key rotation on a recurring schedule whereas the CA and leaf TTL configurations can be updated as part of the initial deployment.

5.4.1 Rotate gossip key

Consul uses a 256 bit symmetric gossip encryption key to protect Serf traffic between Consul client agents. To prevent the risk of exposure, you should periodically rotate this key based on your organization's existing encryption key rotation standards.

5.4.2 Rotate ca certificate

A certificate rotation process is important to protect the integrity of traffic flowing between your Consul Enterprise servers and any client agents. Sensitive information that may be exchanged can include things like ACL tokens or routing information. If a certificate is ever compromised, having a rotation schedule ensures that any attacker is only able to intercept or tamper with traffic for as long as the certificate is valid for. We recommend that you leverage the Vault Enterprise PKI secrets engine to automate the rotation and distribution of your Consul Enterprise server certificates. Future HVD guides will include more prescriptive information around leveraging Vault with Consul Enterprise.

5.4.3 Certificate TTLs

When leveraging auto encrypt for client certificate distribution, you may want to consider adjusting the expiration times for the Root CA, as well as any leaf certificates to align with security practices defined within your organization. The shorter the lifetime of any leaf certificates or CA, the less risk that they can be used for any extended time period if compromised. The default values should work for most organizations, which is 10 years for the CA certificate, and 3 days for the leaf certificates.

Retrieve the current CA configuration.

```
$ curl http://127.0.0.1:8500/v1/connect/ca/configuration
{
    "Provider": "consul",
    "Config": {
        "LeafCertTTL": "72h",
        "IntermediateCertTTL": "8760h",
        "RootCertTTL": "87600h"
    },
    "ForceWithoutCrossSigning": false
}
```

Update the CA configuration.

```
# payload.json
{
    "Provider": "consul",
    "Config": {
        "LeafCertTTL": "72h",
        "IntermediateCertTTL": "8760h",
        "RootCertTTL": "87600h"
    },
    "ForceWithoutCrossSigning": false
}
```

```
$ curl --request PUT --data @payload.json http://127.0.0.1:8500/v1/connect/ca/
    configuration
```

6 Observability and alert management

Ensuring the health and performance of Consul and the services it supports is essential. Monitoring and observability provide the insights needed to understand system behavior, enabling proactive

management and issue prevention.

Consul emits a wealth of telemetry and logs, offering in-depth visibility into the platform's functionality, and the services running on it. While Consul provides basic tools and UI to view these metrics, it is recommended to leverage feature-rich monitoring and observability tools such as Datadog, AppDynamics CNS, Grafana, and others. HashiCorp has partnered with industry-leading APM vendors to create integrations that make it easy to monitor Consul and its services as part of centralized monitoring and observability tools.

Before starting implementation, building a comprehensive observability strategy is vital to success. This strategy should encompass all teams—platform, DevOps, network, and security teams. It is crucial to select scalable and adaptable solutions that can meet growing demands.

HashiCorp advises developing a thorough plan to build and implement an observability strategy. This plan should align with both current objectives and future needs, ensuring it scales as your Consul usage grows.

6.1 Prerequisites

We recommend that you have completed the following steps before implementing the guidance in this document:

1. A fully functional, production-ready Consul cluster (LTS version 1.15.0 or higher)
2. Services registered and discovered in Consul
3. A fully deployed observability tool, such as Datadog

6.2 Build your observability strategy

To successfully implement a comprehensive observability solution, take a holistic approach. This involves thorough evaluation and standardization of the following elements:

- Strategy
- Personas
- Observability pillars
- Alerts and notifications
- Dashboards

- Processes
- Integrations
- Rollout strategy
- Project plan

6.3 Strategy

Begin by defining what success looks like. This involves understanding the needs of different stakeholders, selecting appropriate tools and processes, and aligning these elements with your business objectives.

6.3.1 Vision

As with any successful project or initiative, it is important to have an overarching vision for your observability needs. This vision will help guide teams and priorities, ensuring alignment with the overall goals.

6.3.2 Use cases

Engage with all stakeholders to identify and prioritize use cases based on business objectives. Here are a few examples:

- **Proactive monitoring:** Detect and address issues before they impact end users.
- **Troubleshooting:** Provide end-to-end visibility, and meaningful insights across applications, services, and infrastructure to aid in troubleshooting.
- **Security and compliance: **Provide data and insights for security and compliance assessments/audits.
- **Capacity planning:** Facilitate capacity planning for resources and infrastructure needed to support business growth.

6.3.3 Objectives

Establishing clear objectives that align with the needs and goals of your organization is crucial. Here are some observability objectives to consider:

Proactive monitoring

- Proactive issue detection and resolution
  - **Goal:** Identify and address issues before they impact users.
  - **Key metrics:** Error rates, anomaly detection rates, time to detect and resolve issues.
- Comprehensive visibility across systems
  - **Goal:** Achieve full visibility into the operational state of all components.
  - **Key metrics:** Coverage of monitored components, completeness of logging and tracing, and data granularity.

Troubleshooting

- Improved incident response and root cause analysis
  - **Goal:** Streamline incident response and effectively determine the root cause of issues.
  - **Key metrics:** Incident response time, root cause identification time, post-incident analysis effectiveness.
- Improved system reliability and availability
  - **Goal:** Minimize system downtime and ensure high availability of services.
  - **Key metrics:** Uptime percentage, Mean Time Between Failures (MTBF), Mean Time to Recovery (MTTR).

Security and compliance

- Enhanced security monitoring and compliance
  - **Goal:** Ensure robust security monitoring and compliance with regulatory requirements.
  - **Key metrics:** Security incident detection rate, compliance audit success rate, security monitoring coverage.

Capacity planning

- Enhanced performance monitoring

    - **Goal:** Ensure optimal performance of applications and services.
    - **Key metrics:** Response times, latency, throughput, resource utilization (CPU, memory).
- **Scalability and flexibility of monitoring solutions**
    - **Goal:** Ensure observability solutions can scale with business growth and adapt to changing requirements.
    - **Key metrics:** Scalability benchmarks, ease of integration with new technologies, adaptability to new use cases.

Other goals

Additionally, establish objectives that focus on business expectations, user experience, and organizational improvements. Here are a few such objectives to consider:

- **SLAs/SLOs**
    - **Goal:** Establish Service Level Agreements (SLAs) and Service Level Objectives (SLOs).
    - **Importance:** These are crucial for setting performance baselines, determining alert thresholds, and severities, and establishing appropriate notification methods.
- **User experience optimization**
    - **Goal:** Enhance the end-user experience by ensuring seamless and responsive services.
    - **Key metrics:** User satisfaction scores, application error rates, response times, and service availability from the end user's perspective.
- **Improved collaboration across teams**
    - **Goal:** Facilitate better collaboration and knowledge sharing among development, operations, and security teams.
    - **Key metrics:** Cross-team resolution times, number of collaborative incidents, shared documentation, and practices.

6.4 Personas

Observability solutions should be designed to cater to various personas within your organization. Identifying these relevant personas will help determine why they need the solution, how and when they will interact with it, and what information they will gather to address their issues. Here are some common personas to consider:

- Developers
- Platform engineers
    - Infrastructure specialists
    - DevOps engineers
    - Security architects
    - SREs
- Security engineers
- Network operators
- Cloud engineers
- Service owners
- Management

Identifying personas enables you to standardize various aspects of your observability strategy and implementation, such as—

- **Role-Based Access Control (RBAC):** Develop RBAC standards based on personas to ensure only the intended teams/people have access to create, update, delete, or view dashboards, alerts, and notifications. Not all personas need access to everything.
- **Alerts and notifications:** Design persona-based alerts and notifications to keep noise low and ensure relevant information reaches the right people.
- **Dashboards:** Create persona-based dashboards to speed up the troubleshooting process by presenting individuals with data and information relevant to them.

By tailoring your observability strategy to the specific needs and roles of different personas, you can enhance the effectiveness of your monitoring and incident response processes.

7 Observability pillars

When monitoring Consul, it is helpful to tackle it based on the two primary components:

Monitoring the control plane

- This includes your Consul servers, which manage the cluster's overall operations. These servers can run on bare metal, virtual machines (VMs), or within Kubernetes environments. However, it is strongly recommended to deploy them on VMs, following HashiCorp's recommended deployment architecture.

Monitoring the data plane

- This involves observing the Consul clients and associated elements. Key components include Consul agents (clients), sidecar proxies, Consul dataplanes (when operating within containerized environments like Kubernetes), and Gateways, which include Mesh, API, and Terminating Gateway.

For both the control plane and data planes, it is essential to establish standards across the three core pillars of observability: metrics, logs, and traces. This approach ensures comprehensive monitoring is configured, which provides visibility for effectively managing and troubleshooting your Consul environment.

7.1 Monitor the control plane

7.1.1 Metrics

Consul emits a significant amount of telemetry metrics, which can be overwhelming. To start, focus on The Four Golden Signals of monitoring:

1. **Latency**: The time it takes to service a request.
2. **Traffic**: The volume of requests a system handles at any given time.
3. **Errors**: The rate of failed requests, either explicitly (e.g., HTTP 500s), implicitly (e.g., HTTP 200 success response with incorrect content), or by policy (e.g., any request over a committed response time).
4. **Saturation**: The percentage of resources consumed, indicating how full your system is.

Key metrics to monitor

- System metrics

  Visibility into system resource usage helps understand server saturation. Key resources include:

  - CPU
  - Memory
  - Disk

  Refer to "Server host metrics" for specific metrics for the above resources

- Consul metrics

  Server health: Please refer to the Consul documentation "Agent telemetry - server health" for details.

Enable Consul telemetry

Configure each Consul agent (server) to expose and capture local agent metrics with Prometheus and DogstatsD:

```
telemetry = {
 prometheus_retention_time = "1h"
 dogstatsd_addr            = "127.0.0.1:8125"
 disable_hostname = true
}
```

> Note
>
> You may need to restart the Consul agent for the configuration to take effect. Follow best practices and restart the leader node last.

Recommendations

- Recommended server metrics to monitor:

  - Transaction timing
  - Leadership changes
  - Certificate authority expiration
  - Autopilot

- Memory usage
- Garbage collection
- Network activity - RPC count
- Raft thread saturation
- Raft replication capacity issues
- License expiration
- Bolt DB performance

For full details on the specific metrics refer to "Agent telemetry - server health"

- Additionally, monitor the following metrics:
  - `consul.rpc.*` - specifically, `consul.rpc.query` and `consul.rpc.queries_blocking` are the two most relevant.
  - `consul.grpc.server.*` - specifically, `consul.grpc.server.stream.count`, and `consul.grpc.server.streams` show the number of streams being processed by the server.
  - `consul.xds.server.streams` - number of xDS streams. In version 1.14 and higher, this metric will be a large source of read load.

References

- Datadog, Consul, and Envoy metrics monitoring | VM Edition
- Datadog - Key metrics for monitoring Consul
- Monitor Consul server health and performance with metrics and logs

7.1.2 Logs

Consul generates two types of logs, Consul application logs and audit logs. Each serve distinct purposes for system monitoring and compliance. These logs contain valuable information regarding system interactions and current operational status. Integrating these logs into your Application Performance Monitoring (APM) platform enables the correlation of metrics with log events, facilitating efficient troubleshooting of issues.

Consul logs

By default, Consul generates application logs and outputs to `stdout`. Consul logs capture information about the internal state of Consul, such as errors and messages. This information can be valuable in troubleshooting issues. The logs can be redirected to `syslog` or a file.

To redirect Consul logs to a file, add the following to your agent configuration:

```
log_file = "/var/log/consul/consul.log"
log_level = "INFO"
log_rotate_bytes = 52428800
log_rotate_duration = "24h"
log_rotate_max_files = 7
```

Recommendations

- Configure Consul logs to `file`.
- Configure log rotation by setting appropriate values for `log_rotate_bytes`, `log_rotate_duration`, and `log_rotate_max_files`.
- Limit who has access to the logs by setting appropriate file permissions.
- Configure your APM tool to ingest the logs from all Consul agents (e.g. "Datadog - enable log collection").

References

- Consul log configuration parameters

Audit logs

Consul can generate audit logs if enabled. This is an Enterprise feature, so you will need a valid license file. Actionable logs of authenticated events (both attempted and committed) that Consul processes via its HTTP API are captured in these logs. These events are then compiled into a JSON format for easy export and contain a timestamp, the operation performed, and the user who initiated the action.

Audit logging enables security and compliance teams within an organization to get greater insight into Consul access and usage patterns.

To enable audit logging, add the following to your agent configuration:

```
audit {
  enabled = true
  sink "My sink" {
    type   = "file"
    format = "json"
    path   = "data/audit/audit.json"
    delivery_guarantee = "best-effort"
    rotate_duration = "24h"
    rotate_max_files = 15
    rotate_bytes = 25165824
  }
}
```

> **Note**
>
> At least one of rotate_duration or rotate_bytes must be configured to enable audit logging.

To learn more about configuring and interpreting audit logs, please refer to the "Capture Consul events with audit logging" tutorial.

Recommendations

- Configure log rotation by setting appropriate values for `rotate_duration`, `rotate_max_files`, and `rotate_bytes`.
- Limit who has access to audit logs.
- Configure your APM tool to ingest audit logs from all Consul agents (e.g. "Datadog - enable log collection").

References

- Audit logging
- Audit log configuration parameters

7.1.3 Traces

Traces provide a view of the traffic flow from one service to another as users and applications interact with the system. In the context of Consul, it involves capturing layer 7 metrics from the sidecar proxy running on the Consul dataplane. These metrics offer visibility into the flow of traffic, errors, and response times, helping to identify issues and bottlenecks.

Since this primarily relates to the dataplane, we will cover it in detail in the "Monitor the data plane" section below.

> Tip
>
> This is applicable only if Consul's Service Mesh feature is enabled.

7.2 Monitor the data plane

7.2.1 Metrics

Key metrics to monitor

- System metrics

  Similar to the system metrics from Consul servers, it is important to monitor the system resources of your data plane nodes.

  - CPU
  - Memory
  - Disk

  Refer to server host metrics for the specific metrics for the above resources.

- Consul metrics

  The data plane comprises Consul agents (clients), sidecar proxies, and gateways (mesh, ingress, and terminating).

  - Consul agent metrics - Similar to the Consul server agent, agents running as clients emit telemetry data - Refer to metrics reference.

- Gateways (mesh, API, terminating)

  Gateways also emit telemetry related to the network traffic passing through them. Metrics include ingress/egress request details, errors, and performance information.

- Consul dataplane

  When running in containerized environments, Consul supports a lightweight agent called Consul dataplane, which also emits telemetry. Refer to "Consul dataplane telemetry" for more information.

- Services and health checks

  Additionally, monitor the services running in the Consul datacenter. These include:

  - Catalog

    Use the `catalog` command to query all registered services in a Consul data center.

  - Service endpoints

    Use the `/agent/service/:service_id` API endpoint to query individual services.

  - Health checks

    Use the local service healthcheck to get the aggregated state of service(s) on the local agent by name

- Network

  In a microservices environment, the real-time traffic and health of the network is critical, so monitoring it is valuable. APM tools like Datadog provide a robust network monitoring solution (NPM) which should be leveraged.

  - Datadog: Refer to "Datadog NPM now supports Consul networking" for more details.

Enable Consul telemetry

Configure each Consul agent (client) to expose and capture local agent metrics with Prometheus and DogstatsD:

```
telemetry = {
 prometheus_retention_time = "1h"
 dogstatsd_addr            = "127.0.0.1:8125"
 disable_hostname = true
}
```

> Note
>
> You may need to restart the Consul agent for the configuration to take effect.

Proxy monitoring

Consul proxy metrics provide detailed health and performance information about your service mesh applications. This includes upstream/downstream network traffic metrics, error rates, and additional performance information that you can use to understand your distributed applications.

Once proxy metrics are enabled in Consul, you do not need to configure or instrument your applications in the service mesh to leverage proxy metrics.

Enabling Envoy sidecar proxy metrics

This can be configured in two ways:

1. Global default, which applies to all Envoy proxies in the Consul datacenter.

```
# File: proxy-defaults.hcl
Kind        = "proxy-defaults"
Name        = "global"
Config      = {
 envoy_prometheus_bind_addr = "127.0.0.1:9102"
 envoy_dogstatsd_url        = "udp://127.0.0.1:8125"
}
```

Refer to "Proxy defaults configuration reference" documentation for more details.

1. Service (sidecar) specific, which applies to the specifically configured proxy.

```
# web.hcl - web service and sidecar proxy registration
service {
 name = "web"
 id   = "web-1"
 port = 8080

 connect {
   sidecar_service{
     proxy = {
       config = [
         {
             envoy_prometheus_bind_addr = "127.0.0.1:9102"
            envoy_dogstatsd_url        = "udp://127.0.0.1:8125"
         }
       ]
     }
   }
 }
```

For the configured changes to take effect, restart Envoy, or re-generate its bootstrap configuration

using the `consul connect envoy` command

Recommendations

For full end-to-end visibility, HashiCorp recommends collecting telemetry from all components of your service mesh namely, agents (client), sidecar proxy (envoy proxy), gateways (envoy proxy), and Consul data plane.

References

- Monitor application health and performance with Consul proxy metrics
- Consul dataplane telemetry
- Configure metrics for Consul on Kubernetes
- Monitoring service-to-service communication with Envoy

7.2.2 Logs

Similar to the Consul control plane, the data plane components also generate both application logs, and access logs. The primary components are the Consul agents (clients).

Consul logs

The Consul agent (client) running in your Consul data center generates logs to `stdout` by default. Agents can be configured to redirect logs to `syslog` or a file.

To redirect Consul logs to a file, add the following to all of your agent (client) configurations:

```
log_file = "/var/log/consul/consul.log"
log_level = "INFO"
log_rotate_bytes = 52428800
log_rotate_duration = "24h"
log_rotate_max_files = 7
```

Recommendations

- Enable logs from all components of your data plane.
- Configure log rotation by setting appropriate values for `log_rotate_bytes`, `log_rotate_duration`, and `log_rotate_max_files`.

- Limit who has access to the logs by setting appropriate file permissions.
- Configure your APM tool to ingest the logs from all Consul agents (e.g. "Datadog - enable log collection").

References

- Monitor Consul server health and performance with metrics and logs

Access logs

Envoy proxy can emit access logs, which record application connections and requests that pass through proxies in a service mesh, including sidecar proxies and gateways.

This can help in troubleshooting issues, threat detection, and audit compliance.

- Sidecar proxy can be configured to emit access logs. These logs record service-to-service connections and requests in a service mesh.

  It is configured globally as follows:

```
Kind      = "proxy-defaults"
Name      = "global"
AccessLogs {
   Enabled = true
}
```

- Gateway logs

  Like the sidecar proxy (Envoy), gateways (mesh, API, and terminating) can emit access logs to record network traffic that passes through them in a service mesh.

Recommendations

- Enable access logs in JSON format so APM tools can ingest them.
- Envoy proxy does not handle log rotation for the access logs it generated. Use an appropriate log rotation tool to periodically rotate logs.

References

- Access logs
- Debug service mesh events and errors with Consul proxy access logs

Audit logs

Consul agents that run as clients in your data plane also generate audit logs. Like your Consul servers, configure all your Consul client agents to enable audit logging.

To enable audit logging add the following to all your agent (client) configurations:

```
audit {
  enabled = true
  sink "My sink" {
    type   = "file"
    format = "json"
    path   = "data/audit/audit.json"
    delivery_guarantee = "best-effort"
    rotate_duration = "24h"
    rotate_max_files = 15
    rotate_bytes = 25165824
  }
}
```

Recommendations

- Configure log rotation by setting appropriate values for `rotate_duration`, `rotate_max_files`, and `rotate_bytes`.
- Limit who has access to audit logs.
- Configure your APM tool to ingest audit logs from all Consul agents.

References

- Audit logging
- Audit log configuration parameters
- Capture Consul events with audit logging

7.2.3 Traces

Traces provide a comprehensive and continuous view of an application's network traffic, aiming to follow a request flow as it moves from one service to another. Think of tracing as representing a single user's journey through an entire app stack. Its primary purpose is optimization rather than being reactive. By tracing through a stack, developers can pinpoint errors or performance bottlenecks.

When issues arise, tracing allows you to understand how the user encountered the problem by examining:

- Which function was involved
- The function's duration
- Parameters passed
- How deep into the function the user progressed

The two parts of traces

1. **Trace**: Represents the entire journey of a request or action as it moves through various nodes of a distributed system, especially containerized applications or microservices architectures.
2. **Span**: An operation or work taking place on a service. For example, a web server responding to an HTTP request or a single invocation of a function. A span has a start time and an end time. A series of tagged time intervals, known as spans, form a single trace in distributed tracing.

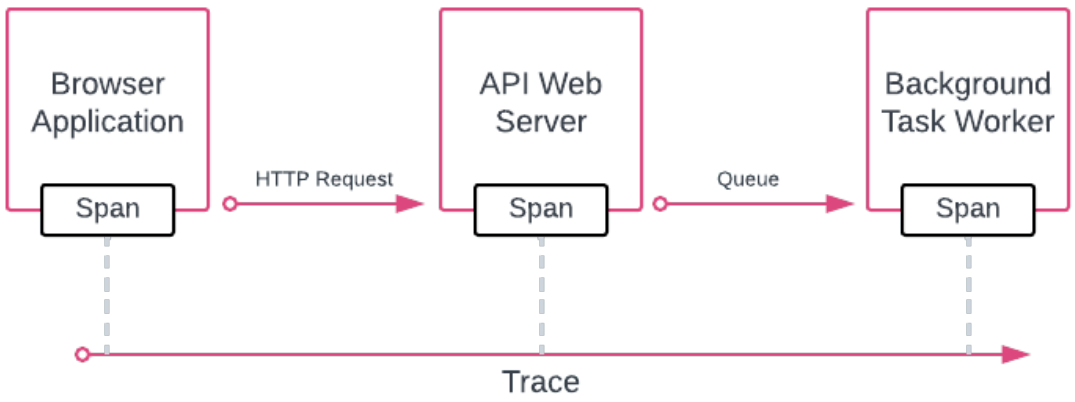


Figure 4: Consul Observability Trace Workflow

> **Note**
>
> Before you can start tracing, your applications must be instrumented with an appropriate tracing library.

Instrument your application with Datadog

Consul does not automatically implement tracing for your applications. They must be instrumented to support tracing.

If Datadog is your APM solution, you can use Datadog's SDK and tracing library (Tracer) to instrument your applications.

For more implementation details and information on Datadog SDK's supported environments and programming languages, please refer to Datadog's "Application Instrumentation" documentation.

Consul trace configuration

This involves configuring your sidecar proxy (Envoy) by adding spans. The following two keys need to be configured:

- `envoy_tracing_json`
- `envoy_extra_static_cluster_json`

For details on trace configuration, please refer to the following articles:

- Consul's distributed tracing
- Envoy tracers

> Tip
>
> Consul does not support all proxies and protocols. Refer to Considerations for details.

Configure proxy defaults so the settings will automatically apply to all proxies by default.

> Note
>
> All proxies must be restarted for the configuration changes to take effect.

Recommendations

- Correlate logs to traces
  - Inject span IDs into your log messages, so your APM tool can correlate them.
    - Datadog: For implementation details please refer to Datadog's "Correlate Logs and Traces" document.

- While implementing the instrumentation of your application, thoroughly test and verify the tracing data:
  - Use Consul's (or other 3rd party tools') fault injection feature to inject errors and timeouts.
  - Stress test your complete environment and verify your tracing implementation is identifying the bottlenecks.
- Data security

  As with telemetry and logging data, tracing data can potentially expose sensitive data, such as personal identifiable information (PII). Be mindful and take measures to obfuscate such data.

  - Datadog: You can either leverage Sensitive Data Scanners or implement Datadog's tracing library to remediate such data before sending it to Datadog.

    Refer to Datadog's Data Security documentation for more information.

References

- Distributed tracing
- Enabling distributed tracing with HashiCorp Consul
- Consul demo - tracing datadog (Demo)
- Distributed tracing with Consul (Video)
- Leveraging HashiCorp Consul and Datadog for service mesh deployments (Video)
- Proxy defaults configuration reference
- Proxy parameters

8 Additional observability elements

8.1 Alerts and notifications

Alerts and notifications are proactive measures to notify teams of abnormal application behavior. This plays a vital role in observability. They ensure that relevant teams are promptly informed of any issues to prevent potential outages.

When designing alerts and notifications, it is crucial to consider personas and establish consistent standards. Here are key aspects to consider when setting up alerting standards:

- **Persona**: Determine who will receive the notifications based on their role and responsibilities.
- **RBAC (Role-Based Access Control)**: Define who can create, update, delete, and manage alert configurations.
- **Metrics to alert on**: Select metrics that are critical to monitor based on your service level agreements (SLAs) and objectives (SLOs).
- **Thresholds**: Set appropriate thresholds that align with performance targets and operational requirements.
- **Notification channels**: Choose effective channels such as Slack, PagerDuty, JIRA, email, etc., to ensure alerts reach the right teams promptly.

Recommendations

- **Persona based alerts**: Customize alerts with relevant titles, thresholds, severity levels, messages, notification channels, and recipients.
- **Warning thresholds**: Set lower thresholds to receive warnings before critical thresholds are breached.
- **Meaningful notification titles**: Use variables when possible to automatically populate alert titles and messages, providing quick insights into the alert trigger.
- **Monitoring messages**: Include detailed incident information and resolution steps, specifying who to contact (e.g., service owner, technical contact) for swift issue resolution.
- **Noise reduction**: Ensure effective alerts by minimizing noise—avoid excessive alerting or notifications.
- **Flexible alert management**: Provide the ability to disable alerts during troubleshooting or maintenance windows as needed.

Refer to Consul's agent telemetry documentation for guidance on key Consul metrics and thresholds that are essential for configuring effective alerts.

Refer to Consul's "Monitoring service-to-service communication with Envoy" documentation for guidance on key Envoy metrics and alerting thresholds.

References

- Consul monitoring and alerts recommendations
- Agent telemetry

8.2 Dashboards

Managing dynamically changing systems—both services and infrastructure—poses increasing challenges. Observability through visualization and dashboards offers a unified view of your entire IT landscape, enabling effective monitoring, and troubleshooting of applications and infrastructure.

Observability dashboards provide a robust solution by offering—

- Real-time insights into application and infrastructure health
- Centralized visibility across distributed systems
- Correlation of metrics, logs, and traces for rapid issue identification

Recommendations

- Create persona-based dashboards tailored to meet the specific needs of various roles within your organization:
  - **SRE**: Broad insights into services, infrastructure, and networking.
  - **DevOps**: Deeper insights into services, infrastructure, and networking, particularly in response to changes like deployments, feature rollouts, and upgrades.
  - **Developers**: Application and service-specific dashboards.
  - **Network team**: Detailed metrics related to network performance and issues.
  - **Management**: Overall service health and monitor SLA/SLO compliance.
- Based on your chosen visualization solution, leverage vendor-provided dashboards for Consul such as Datadog or Grafana, and customize them to meet your specific requirements.

By tailoring dashboards to the distinct needs of different personas, you ensure that each team member has the visibility necessary to perform their roles effectively.

8.3 Processes

Having insights into the health of your services is crucial, but timely and appropriate actions are necessary to address any issues. Established processes ensure that individuals and teams know exactly what steps to follow for enabling monitoring, and for efficient issue resolution.

Recommendations

Processes to implement:

- **Establish baselines**: Develop and agree upon baseline metrics with service owners, determining how and when these baselines will be established.
- **In-take requests**: Define how service owners should communicate their monitoring requirements to the platform team for configuration, including metrics, baselines, alerts, notifications, and dashboards.
- **Incident response**: Outline procedures for teams to follow during an incident, including how to inform customers, steps for troubleshooting, communication protocols with other teams, and escalation to management.

By having clear, standardized processes, you ensure a coordinated and efficient response to any issues that may arise, minimizing downtime and maintaining service quality.

8.4 Integrations

HashiCorp recommends integrating Consul with third-party APM tools to achieve end-to-end observability of your Consul control plane, Consul data plane, and all registered services. To facilitate this, HashiCorp has partnered with industry-leading APM vendors to provide simple-to-deploy integrations.

Integrating with your existing automation (CI/CD pipelines) is highly recommended.

Recommendations

- **Consul integrations**:
  - Datadog

    - Cisco AppDynamics
    - Grafana
    - Dynatrace

- **Automation**: Use automation tools like Terraform and Ansible to enable/configure monitoring as part of your infrastructure and service deployment process, typically within your CI/CD pipeline. This approach ensures that monitoring is consistent, seamlessly integrated, and operational from the moment your services are deployed.

By leveraging these integrations and automation tools, you can enhance your observability strategy and maintain comprehensive visibility across your systems even when new systems and services come online.

8.5 Rollout strategy

To minimize unexpected disruption it is recommended to establish a clear rollout strategy. This approach helps introduce observability-related changes in a controlled manner, reducing the risk of production issues.

Recommendations

- Take a phased approach:
    - Pilot phase:
        - Begin with a small-scale implementation focused on critical applications and infrastructure.
        - Evaluate the implementation and refine it based on feedback.
    - Full rollout:
        - Gradually expand the observability solution across all systems and teams.
        - Ensure proper documentation and training are provided.
    - Continuous monitoring and improvement:
        - Regularly review the performance of your observability baseline and processes.
        - Incorporate changes as they become available through the feedback process.

8.6 Project plan

To effectively implement your observability strategy, HashiCorp recommends structuring it as a dedicated project. Develop a comprehensive project plan that outlines all tasks, assigns owners, identifies dependencies, and sets clear timelines. This structured approach ensures efficient execution and timely completion of the project.

Example project plan

1. Project initiation phase
    - Define project scope, objectives, and success criteria.
    - Identify stakeholders and establish communication channels.
    - Assign project manager and core team members.
2. Planning phase
    - Conduct a thorough assessment of current observability capabilities.
    - Define SLAs/SLOs and establish baseline metrics.
    - Develop detailed requirements for metrics, logs, and tracing implementations.
    - Identify the observability and log management tools, as well as any auxiliary tools required to meet your objectives
    - Create a project timeline and milestone schedule.
3. Implementation phase
    - Deploy and configure observability tools and platforms (e.g., Datadog, Prometheus, Grafana, Jaeger).
    - Integrate monitoring agents and instrumentation into Consul control and data planes.
    - Set up centralized logging and implement distributed tracing frameworks.
4. Testing and validation phase
    - Conduct functional testing to ensure monitoring tools capture expected metrics.
    - Validate logging configurations and trace propagation across Consul components.

- Perform load testing and simulate failure scenarios to verify alerting and response mechanisms.

5. Deployment and rollout phase
    - Plan and execute phased deployment of observability enhancements.
    - Provide training sessions for operations and support teams on new monitoring capabilities.
    - Monitor system performance post-deployment and address any immediate issues.
6. Monitoring and optimization phase
    - Establish ongoing monitoring processes and routines.
    - Continuously optimize metrics collection, logging practices, and tracing configurations.
    - Conduct regular reviews to refine SLAs/SLOs based on operational insights.
7. Documentation and knowledge sharing
    - Document project outcomes, including configurations, processes, and lessons learned.
    - Share best practices and operational guidelines with relevant teams.
    - Update documentation as new tools and practices are adopted.

By following this structured project plan, your organization can effectively implement HashiCorp's recommended observability strategy, ensuring robust monitoring and operational efficiency across your Consul environment.

8.7 Conclusion

Consul provides powerful tools for service discovery, health monitoring, and secure communication that integrate seamlessly into a comprehensive observability stack. Tailoring your strategy to include these features will ensure you maintain robust visibility and control over your applications, services, and infrastructure as your company grows.

8.8 Platform monitoring recommendations from initial configuration

8.8.1 Configure platform monitoring

After setting up your first datacenter, it is an ideal time to make sure your deployment is healthy and establish a baseline. Here are recommendations for monitoring your Consul control and data plane. By keeping track of these components and setting up alerts, you can better maintain the overall health and resilience of your service mesh.

Consul control plane monitoring

The Consul control plane consists of the following components:

- RPC communication between Consul servers and clients.
- Gossip Traffic: LAN
- Consul cluster peering

It is important to monitor and establish baseline and alert thresholds for Consul control plane components for detection of abnormal behavior. Note that these alerts can also be triggered by some planned events like Consul cluster upgrades, configuration changes, or leadership change.

To help monitor your Consul control plane, we recommend to establish a baseline and standard deviation for the following:

- Server health
- Leadership changes
- Key metrics
- Autopilot
- Network activity
- Certificate authority expiration

We recommend monitoring the following parameters for Consul agents' health:

- Disk space and file handles
- RAM utilization
- CPU utilization
- Network activity and utilization

We recommend using an application performance monitoring (APM) system to track these metrics. For a full list of key metrics, visit the Key metrics section of Telemetry documentation.

Host-level alerts

When collecting metrics, it is important to establish a baseline. This baseline ensures your Consul deployment is healthy, and serves as a reference point when troubleshooting abnormal Cluster behavior. Once you have established a baseline for your metrics, use them and the following recommendations to configure reasonable alerts for your Consul agent.

Memory

Set up an alert if your RAM usage exceeds a reasonable threshold (for example, 90% of your allocated RAM). Refer to the Memory Alert Recommendations for further details.

Metrics to monitor:

Metric Name	Description
`consul.runtime.alloc_bytes`	Measures the number of bytes allocated by the Consul process.
`consul.runtime.sys_bytes`	The total number of bytes of memory obtained from the OS.
`mem.total`	Total amount of physical memory (RAM) available on the server.
`mem.used_percent`	Percentage of physical memory in use.
`swap.used_percent`	Percentage of swap space in use.

Consul agents are running low on memory if `consul.runtime.sys_bytes` exceeds 90% of mem.total_bytes, mem.used_percent is over 90%, or `swap.used_percent` is greater than 0. You should increase the memory available to Consul if any of these three conditions are met.

CPU

Set up an alert to detect CPU spikes on your Consul server agents. Refer to the CPU Alert Recommendations for further details.

Metrics to monitor:

Metric Name	Description
`cpu.user_cpu`	Percentage of CPU being used by consul processes
`cpu.iowait_cpu`	Percentage of CPU time spent waiting for I/O tasks to complete.

If `cpu.iowait_cpu` is greater than 10%, it should be considered critical as Consul is waiting for data to be written to disk. This could be a sign that Raft is writing snapshots to disk too often.

Network

The data sent between all Consul agents must follow latency requirements for total round trip time (RTT):

- Average RTT for all traffic cannot exceed 50ms. RTT for 99 percent of traffic cannot exceed 100ms.
- Set an alert to detect when the RTT exceeds these values. These values should be compared to the host's network latency so the RTT does not exceed these values.
- Refer to the Reference architecture to learn more about network latency and bandwidth guidance.

Metrics to monitor:

Metric Name	Description
`net.bytes_recv`	Bytes received on each network interface.
`net.bytes_sent`	Bytes transmitted on each network interface.

Sudden increases to the net metrics, greater than 50% deviation from baseline, indicates too many requests that are not being handled.

File descriptors

Metrics to monitor:

Metric Name	Description
`linux_sysctl_fs.file-nr`	Number of file handles being used across all processes on the host.
`linux_sysctl_fs.file-max`	Total number of available file handles.

By default, process and kernel limits are conservative, you may want to increase the limits beyond the defaults. If the `linux_sysctl_fs.file-nr` value exceeds 80% of `linux_sysctl_fs.file-max`, the file handles should be increased.

Disk activity

Metrics to monitor:

Metric Name	Description
`diskio.read_bytes`	Bytes read from each block device.
`diskio.write_bytes`	Bytes written to each block device.
`diskio.read_time`	Time spent reading from disk, in cumulative milliseconds.
`diskio.write_time`	Time spent writing to disk, in cumulative milliseconds.

Sudden, large changes to the disk I/O metrics, greater than 50% deviation from baseline or more than 3 standard deviations from baseline indicates Consul has too much disk I/O. Too much disk I/O can cause the rest of the system to slow down or become unavailable, as the kernel spends all its time waiting for I/O to complete.

Raft monitoring

Consul uses Raft for consensus protocol. High saturation of the Raft goroutines can lead to elevated latency in the rest of the system and may cause the Consul cluster to be unstable. As a result, it is important to monitor Raft to track your control plane health. We recommend the following actions to keep control plane healthy:

- Create an alert that notifies you when Raft thread saturation exceeds 50%.
- Monitor Raft replication capacity when Consul is handling large amounts of data and high write throughput.
- Lower raft_multiplier to keep your Consul cluster stable. The value of raft_multiplier defines the scaling factor for Consul. Default value for raft_multiplier is 5.A short multiplier minimizes failure detection and election time but may trigger frequently in high latency situations. This can cause constant leadership churn and associated unavailability. A high multiplier reduces the chances that spurious failures will cause leadership churn but it does this at the expense of taking longer to detect real failures and thus takes longer to restore Consul cluster availability.Wide networks with higher latency will perform better with larger raft_multiplier values.

Raft protocol health

Metrics to monitor:

Metric Name	Description
`consul.raft.thread.main.saturation`	An approximate measurement of the proportion of time the main Raft goroutine is busy and unavailable to accept new work.
`consul.raft.thread.fsm.saturation`	An approximate measurement of the proportion of time the Raft FSM goroutine is busy and unavailable to accept new work.
`consul.raft.fsm.lastRestoreDuration`	Measures the time taken to restore the FSM from a snapshot on an agent restart or from the leader calling installSnapshot.
`consul.raft.rpc.installSnapshot`	Measures the time taken to process the installSnapshot RPC call. This metric should only be seen on agents which are currently in the follower state.

Metric Name	Description
`consul.raft.leader.oldestLogAge`	The number of milliseconds since the oldest entry in the leader's Raft log store was written. In normal usage this gauge value will grow linearly over time until a snapshot completes on the leader and the Raft log is truncated.

Saturation of consul.raft.thread of more than 50% can lead to elevated latency in the rest of the system and cause cluster instability.

Snapshot restore happens when a Consul agent is first started, or when specifically instructed to do so via RPC. For Consul servers, consul.raft.fsm.lastRestoreDuration tracks the duration of the operation. For Consul clients, from the leader's perspective when it installs a new snapshot on a follower, `consul.raft.rpc.installSnapshot` tracks the timing information. Both these metrics should be consistent, without sudden large changes.

Plotting of `consul.raft.leader.oldestLogAge` should look like a saw-tooth wave increasing linearly with time until the leader takes a snapshot and then jumping down as the oldest Raft logs are truncated. The lowest point on that line should remain comfortably higher (for example, 2x or more) than the time it takes to restore a snapshot.

BoltDB monitoring

Metrics to monitor:

Metric Name	Description
`consul.raft.boltdb.storeLogs`	Measures the amount of time spent writing Raft logs to the db.
`consul.raft.boltdb.freelistBytes`	Represents the number of bytes necessary to encode the freelist metadata.
`consul.raft.boltdb.logsPerBatch`	Measures the number of Raft logs being written per batch to the db.

Metric Name	Description
`consul.raft.boltdb.writeCapacity`	Theoretical write capacity in terms of the number of Raft logs that can be written per second.

Sudden increases in the `consul.raft.boltdb.storeLogs` times will directly impact the upper limit to the throughput of write operations within Consul.

If the free space within the database grows excessively large, such as after a large spike in writes beyond the normal steady state and a subsequent slow down in the write rate, then Bolt DB could end up writing a large amount of extra data to disk for each Raft log storage operation. This will lead to an increase in the `consul.raft.boltdb.freelistBytes` metric - a count of the extra bytes that are being written for each Raft log storage operation beyond the Raft log data itself. Sudden increases in this metric can be correlated to increases in the `consul.raft.boltdb.storeLogs` metric indicating an issue.

The maximum number of Raft log storage operations that can be performed each second is represented with the `consul.raft.boltdb.writeCapacity` metric. When Raft log storage operations are becoming slower you may not see an immediate decrease in write capacity due to increased batch sizes of each operation. Sudden changes in this metric should be further investigated.

The `consul.raft.boltdb.logsPerBatch` metric keeps track of the current batch size for Raft log storage operations. The maximum allowed is 64 Raft logs. Therefore if this metric is near 64 and the `consul.raft.boltdb.storeLogs` metric is seeing increased time to write each batch to disk, it is likely that increased write latencies and other errors may occur.

Consul dataplane monitoring

Consul data plane collects metrics about its own status and performance. Consul data plane uses the same external metrics store that is configured for Envoy. To enable telemetry for Consul data plane, enable telemetry for Envoy by specifying an external metrics store in the proxy-defaults configuration entry or directly in the `proxy.config` field of the proxy service definition.

Service monitoring

You can extract the following service-related information:

- Use the catalog command or the Consul UI to query all registered services in a Consul datacenter.
- Use the /agent/service/:service_id API endpoint to query individual services.

9 Backup and restore operations

This page covers recurring backup validation and restore operations. Initial backup tooling configuration belongs in the Installation Guide.

9.0.1 Backup Consul

You should regularly test and validate the backup and restore processes in a non-production environment to ensure you are well prepared if you need to perform a restore against production systems and ensure your ability to meet your defined recovery time objectives and recovery point objectives. Encrypt your snapshot backups to comply with organizational and regulatory requirements. Amazon S3 encrypted objects using server-side encryption by default. Ensure you follow any Key management processes defined within your organization when leveraging S3 encrypted backups. Refer to Amazon's documentation for encryption within S3.

Backup cadence with the Consul Enterprise snapshot agent

To automate the backup process, utilize the Consul Enterprise Snapshot Agent. Configure the snapshot agent to take regular snapshots of the Consul server's state and save them to a secure location. Store backups in a different physical/virtual location to prevent data loss during infrastructure failures. You can use Amazon S3 replication techniques to achieve this.

Below is an example configuration that can be used within Amazon to backup snapshots to an S3 bucket.

```
{
    "snapshot_agent": {
        "http_addr": "https://127.0.0.1:8501",
        "ca_file": "/consul/config/tls/consul-agent-ca.pem",
        "token": "$${SNAPSHOT_TOKEN}",
        "snapshot": {
            "interval": "${snapshot_agent.interval}",
            "retain": ${snapshot_agent.retention},
            "deregister_after": "8h"
        },
        "aws_storage": {
            "s3_region": "${aws_region.name}",
            "s3_bucket": "${snapshot_agent.s3_bucket_id}"
        }
    }
}
```

Follow the Installation Guide for your Consul Enterprise deployment into EC2. You can use the exposed variables to directly modify the interval and retention setting to align with your recovery point objectives. The S3 bucket and IAM security are automatically provisioned as part of the Terraform module covered within that guide.

When configuring the snapshot agent manually, you must provide a similar ACL policy.

```
# Required to read and snapshot ACL data
acl = "write"
# Allow the snapshot agent to create the key consul-snapshot/lock, which will
# serve as a leader election lock when multiple snapshot agents are running in
# an environment
key "consul-snapshot/lock" {
    policy = "write"
}
# Allow the snapshot agent to create sessions on the specified node for lock
session_prefix "" {
    policy = "write"
}
# Allow the snapshot agent to register itself into the catalog
service "consul-snapshot" {
    policy = "write"
}
```

9.0.2 Restore Consul

When restoring from a snapshot backup taken with the Consul Enterprise snapshot agent, the state of the entire cluster will be reverted to the time the snapshot was taken. There isn't an easy way to directly pull individual items from this snapshot without restoring it to an isolated cluster and using the API to retrieve individual items.

You can only restore snapshots to the same Consul version in which they were created. For service discovery use cases, the state contained within the clients is less critical. It will be synchronized back to the server catalog using the anti-entropy methods to keep the agent and server states aligned. This means any services registered after the snapshot will reappear in the catalog after the client agent synchronizes its state with the servers or when the client rejoins the server cluster after the restore operation.

It may be helpful to restart client agents where possible to ensure they aren't caching any data that is no longer relevant after the restore operation. Another consideration is that any tokens generated after the snapshot would no longer be valid and may need to be re-issued.

When leveraging cluster peering, the keys exchanged between peers rotate every three days. If you restore a snapshot created before a peering key was rotated, you must re-create the peering relationship using the same name.

Use the `consul snapshot restore` command to restore a snapshot retrieved from S3 using the Amazon CLI or other S3 API tooling that can retrieve objects.

10 Upgrade procedures

This page covers recurring Consul Enterprise upgrade operations after installation.

Consul is a long-running agent on any nodes participating in a Consul cluster. These nodes consistently communicate with each other. As such, protocol level compatibility and ease of upgrades are important to remember when using Consul. After successfully installing and running Consul Enterprise, it's important to understand the backup and restore procedures to ensure data durability and system availability during unforeseen events.

10.0.1 Upgrades

Upgrading Consul Enterprise is a critical operational task that requires careful planning and execution to ensure service availability. All software upgrades and major server configuration changes should be tested in a non-production environment before being promoted to your production environment. This section elaborates on the key considerations when upgrading Consul Enterprise server instances.

Autopilot automated upgrades

The architecture deployed when following the Installation Guide leverages Autopilot automated upgrades to streamline the upgrade process. This feature automatically transitions to a new set of Consul server agents once ready, allowing the existing server agents to serve traffic during the transition.

When operating within an AWS environment, leverage lifecycle hooks to halt the promotion of new Auto Scaling Groups (ASGs) or the teardown of old ones until the lifecycle hook post-validation is completed. Leverage the `UpgradeVersionTag` to perform safe promotion without changing the Consul version. This allows you to adopt an immutable deployment style and use automated upgrade promotion when changing any server configurations to ensure any changes that could affect server availability are validated as part of the upgrade promotion process. The Terraform module recommended as part of the Installation Guide incorporates these automated upgrade features when deploying to EC2 instances.

Rolling upgrades with configuration management tools

In scenarios without AutoPilot, conduct rolling upgrades using traditional configuration management tools. It is imperative to incorporate health checks within this automation to prevent breaking the server quorum and triggering a server outage. The health checks should be robust enough to identify potential issues that could jeopardize the upgrade or the overall system stability.

Validate each server node's health before upgrading the next node in your server pool. This ensures that service remains operational during the upgrade as long as you do not exceed your deployment's failure tolerance. Even when not leveraging the automated upgrades feature, you can still query the Autopilot state to determine your current failure tolerance and health states of individual server nodes as you progress through an upgrade.

```
$ curl https://127.0.0.1:8501/v1/operator/autopilot/state
{
    "Healthy": true,
    "FailureTolerance": 1,
    "OptimisticFailureTolerance": 4,
    "Servers": {
        "5e26a3af-f4fc-4104-a8bb-4da9f19cb278": {},
        "10b71f14-4b08-4ae5-840c-f86d39e7d330": {},
        "1fd52e5e-2f72-47d3-8cfc-2af760a0c8c2": {},
        "63783741-abd7-48a9-895a-33d01bf7cb30": {},
        "6cf04fd0-7582-474f-b408-a830b5471285": {}
    },
    "Leader": "5e26a3af-f4fc-4104-a8bb-4da9f19cb278",
    "Voters": [
        "5e26a3af-f4fc-4104-a8bb-4da9f19cb278",
        "10b71f14-4b08-4ae5-840c-f86d39e7d330",
        "1fd52e5e-2f72-47d3-8cfc-2af760a0c8c2"
    ],
    "RedundancyZones": {
        "az1": {},
        "az2": {},
        "az3": {}
    },
    "ReadReplicas": [
        "63783741-abd7-48a9-895a-33d01bf7cb30",
        "6cf04fd0-7582-474f-b408-a830b5471285"
    ],
    "Upgrade": {}
}
```

Regular upgrade practice

Familiarize yourself with the upgrade procedures and make it a practice to carry out upgrades regularly. This proactive approach helps capitalize on new features and improvements while avoiding the risks of falling significantly behind on versions. While it's important to avoid bleeding-edge versions until they've proven stable in non-production environments, ensure you don't lag to the extent that the version gap introduces substantial risk during upgrades. In production, promote new releases only after they have achieved stability in non-production deployments.

Downgrade considerations

Downgrades are not supported unless you revert the server state to the pre-upgrade version. This implies that a thorough backup and validation process is important before any upgrade activity to ensure you can roll back to the previous version with the active data before the upgrade.

Upgrade clients

When deploying client agents to the nodes running your services, it is important to keep the client version in sync with the server version. Once your server agents have been upgraded to a new version, begin rolling out updates to the client agents by leveraging configuration management or by layering the new Consul software version into your system image and following an immutable deployment process to refresh the base image of all of your nodes.

Upgrade references

For more information about upgrading Consul, refer to General Upgrade Process and Upgrading Specific Versions.

For more information about autopilot, refer to Autopilot and Automate upgrades with Consul Enterprise.

For information about Kubernetes-specific upgrade instructions, refer to Upgrading Consul on Kubernetes components.

11 Multi-cluster and multi-tenant deployments

Note

This page covers operator setup and governance for Consul namespaces, admin partitions, and cluster peering. For service-to-service usage across multiple infrastructures, see the User Guide: Scale services on multiple infrastructures.

Prerequisites

- Bootstrap the ACL system
- Consul Enterprise v1.15+ (LTS) is installed
- Consul ACL is enabled
- An ACL token with `operator=write` and `acl=write` privileges

Tip

It is not recommended to use a token with global-management permissions.

Scope

- The scope of this section is limited to service discovery.

11.1 Consul multi-tenant

Managing a centrally operated Consul platform can present numerous operational challenges as you onboard more customers (applications) to your Consul platform. One key design pattern for scaling shared platforms is *multi-tenancy*: the ability to host and manage multiple customers on a common (shared) infrastructure. Delegating administrative privileges per tenant allows for self-service, thereby reducing dependency on the Consul platform team.

To effectively manage Consul, HashiCorp recommends a multi-tenancy architecture leveraging Consul's multi-tenancy features: *namespaces* and *admin partitions*. This section will cover the details and provide recommendations.

11.1.1 Personas

It is a best practice to identify the key individuals responsible for designing and implementing Consul multi-tenancy, as well as the end-users of the system within your organization.

- Consul platform engineer: manages the health and stability of the Consul platform, including the creation of namespaces and corresponding operator tokens for respective tenants
- Security engineer: defines the roles and policies that namespace operators are allowed to create.
- Application team:
    - Namespace operator: manages a tenant (or namespace) and creates policies, roles, and tokens for application developers
    - Application developer: registers services to Consul

11.2 Consul namespaces

Consul namespaces, an Enterprise feature introduced in version 1.7.0, allows for the isolation of data and configurations between tenants. A tenant can be a team, an application, or a user.

11.2.1 Multi-tenancy with namespaces

Namespace-level isolation eliminates the need for unique resource names, enabling self-service. Consul operators can delegate namespace-level operator privileges, allowing tenants to manage their own namespace without disrupting others.

Implementing namespaces

- Design
  - Use the `default` namespace solely for administering Consul globally. Do not register services here.
  - Determine namespaces based on access privileges, typically mapping to application teams.
  - Consult with your security team to define namespace-level roles and policies, such as operator, read-write users, and read-only users.
- Deployment
  - Refer to the step-by-step guides "Setup secure namespaces" and "Register and discover services within namespaces".
- Limitations

  Consul namespaces offer logical isolation but have limitations that may not meet all requirements:

  - Configurations such as proxy defaults are global and cannot be isolated per tenant. Teams must coordinate on global configuration changes.
  - IPs cannot overlap. While this provides robust isolation, namespaces still require non-overlapping IP ranges between tenants (e.g. Kubernetes IPs).
  - Complete tenant autonomy is not possible.

  These limitations are manageable in small environments but can be cumbersome in large-scale settings. Consul introduced admin partitions to address these issues, which will be explored in the following section.

References

- Consul Enterprise namespaces
- Setup secure namespaces
- Register and discover services within namespaces

11.3 Consul admin partitions

Organizations often deploy multiple Consul clusters to achieve full control and autonomy between tenants. Each cluster would be dedicated to a different team or service. This approach requires each team to manage both the data plane and the control plane independently.

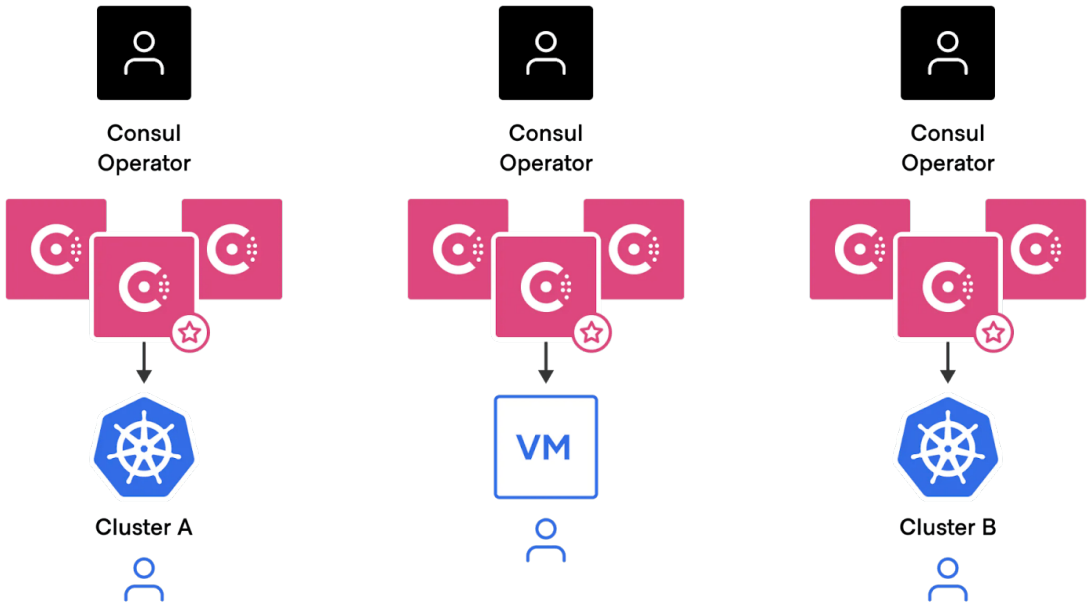


Figure 5: Consul independent admin partitions

While this approach is manageable with a few Consul clusters, it presents significant challenges in large scale deployments. These challenges include management overhead, resource constraints, governance and compliance issues, and high costs.

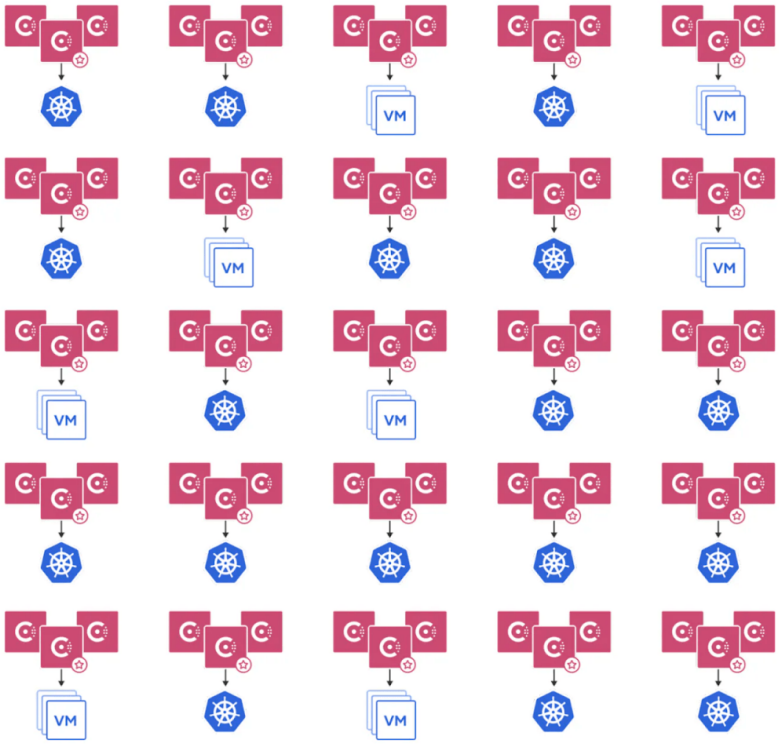


Figure 6: Consul independent admin partitions at scale

11.3.1 Multi-tenancy with admin partitions

Introduced in Consul Enterprise v1.11, admin partitions provide stronger isolation between partitions, addressing the gaps found in namespaces and allowing for complete autonomy.

Consul platform operators can now create multiple partitions within a single data center. These admin partitions are fully isolated from one another while leveraging the same control plane. Each tenant can operate autonomously within an admin partition without the burden of managing the control plane.

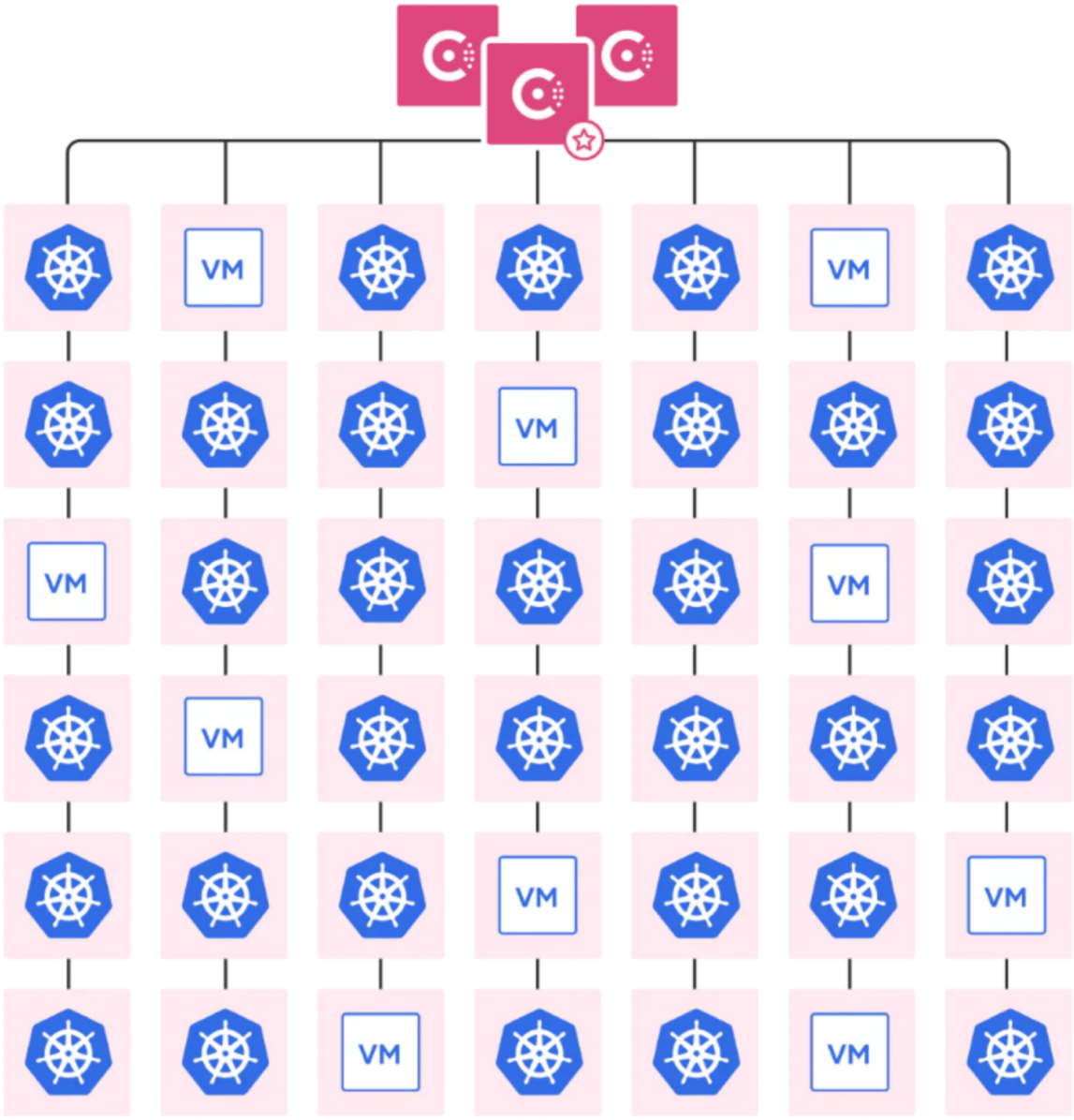


Figure 7: Consul multi-tenancy with admin partitions

Implementing admin partitions

- Design

    - Determine admin partitions based on access privileges, typically mapping to application teams that are run and managed in their own environments or clusters (e.g., VPCs, Kubernetes)
    - Within each admin partition, operators can create multiple namespaces for additional separation to effectively manage their tenant/partition.
    - If services need to be discovered in other admin partitions, use `exported-services`.
    - Leverage Consul sameness groups:
        - Sameness groups designate a set of admin partitions as functionally identical within your network.
        - Adding an admin partition to a sameness group enables Consul to recognize services registered to remote partitions with cluster peering connections as instances of the same service when they share a name and namespace.
- Deployment
    - For detailed instructions on deploying admin partitions in an EKS environment, please refer to the hands-on tutorial titled "Multi-cluster applications with Consul Enterprise admin partitions".

Architecture

The diagram below illustrates an example of a multi-cluster application deployment using admin partitions.

In this setup, the Consul control plane connects to three clusters (data planes):

- **One VM cluster** (Partition A)
- **Two Kubernetes clusters** (Partitions B and C)

Each partition operates independently with its own users, namespaces, KV store, configurations, intentions, and services.

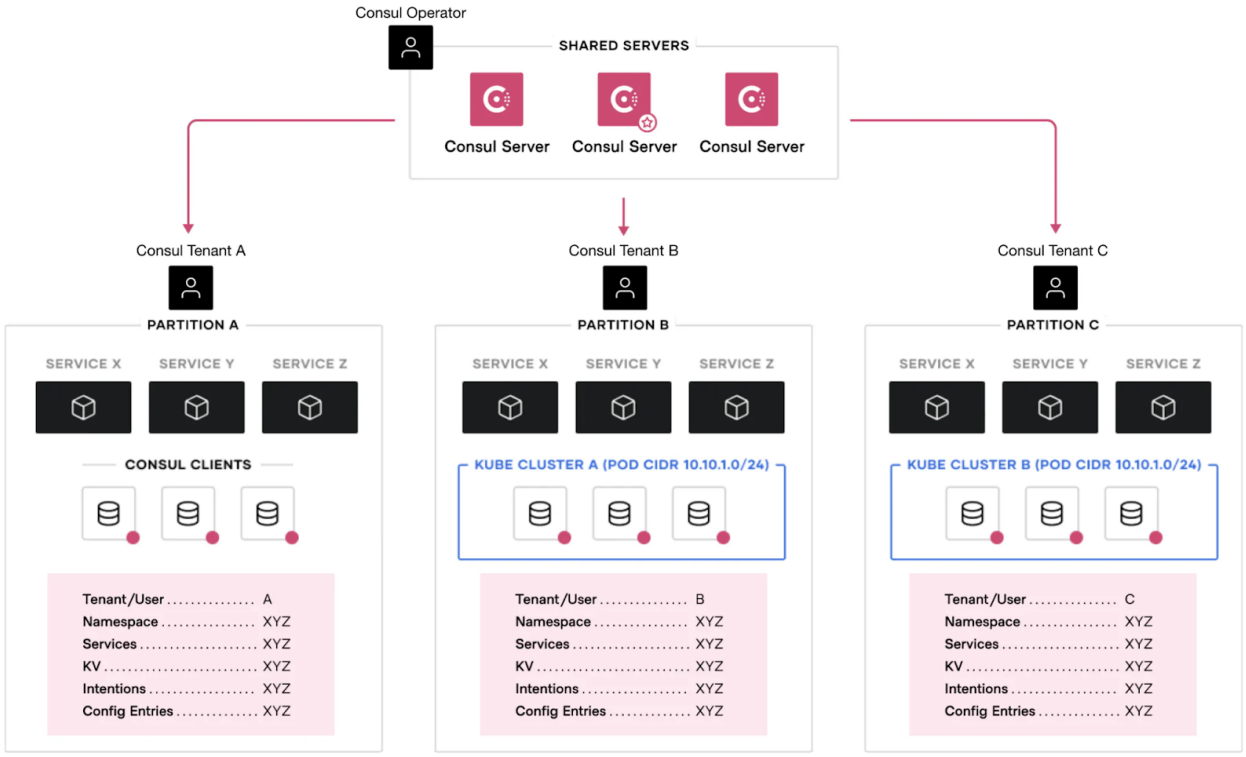


Figure 8: Consul admin partitions with shared services

A complete HashiCorp recommended production deployment architecture can be found in the "Consul: Installation Guide".

References

- Consul Enterprise admin partitions
- Achieving multi-tenancy with admin partitions

11.4 Consul multi-cluster

The use of Consul namespaces along with admin partitions provides the necessary tenant-level isolation to achieve multi-tenancy. However, large enterprises often have additional requirements such as high availability, low latency, governance and compliance, and continued operations under DDIL (Denied, Disrupted, Intermittent, and Limited Impact).

- **High availability**: Ensuring services are highly available and capable of automatic failover across clusters (including cross-region and cross-cloud providers) is critical to minimizing application downtime.
- **Low latency**: For time-sensitive applications, low latency is essential. Co-locating Consul clusters near users may be necessary to achieve this.
- **Governance and compliance**: Regulatory requirements, such as GDPR, mandate that data within Consul clusters remain within national boundaries.
- **Continued operations under DDIL**: Cluster peering additionally provides resilient operations in DDIL environments. This is a critical requirement especially when applied to public sector and even more in the Federal and DoD space. When connectivity and or networking is available, cluster peering is maintained and services are able to be shared while operations continue even when disconnected. This can be extended to quickly peering with mission partners during time of need and safely disconnecting when no longer necessary.

These requirements can be met by connecting or federating multiple Consul data centers, forming Consul multi-cluster architectures.

11.4.1 Cluster peering

As organizations scale, they may need to deploy services across multiple cloud providers and regions. Cluster peering enables the connection of multiple Consul clusters, allowing services in one cluster to securely discover and communicate with services in another.

Consul supports several approaches to connect clusters, broadly categorized into—

- WAN federation
- Cluster peering

For flexibility, reduced complexity, and ease of management, HashiCorp recommends the Consul cluster peering approach. Cluster peering reduces administrative burdens compared to WAN federation. With cluster peering, there is no concept of a primary cluster—which clearly separates administrative boundaries per cluster—ensuring changes in one Consul cluster do not affect peered clusters.

Implementing multi-cluster with Consul cluster peering

- Design
  - Ensure appropriate networking configuration so all Consul clusters are reachable. In AWS, connecting VPCs via AWS Transit Gateway is the recommended approach.
  - Configure Consul mesh gateways in each Consul data center. These gateways will facilitate communication between Consul data centers via the Transit Gateway.
  - For resiliency, configure multiple Consul mesh gateways, preferably through an auto scaling group.
- Deployment
  - Refer to the hands-on tutorial "Connect services between Consul datacenters with cluster peering" for detailed instructions on deploying Consul cluster peering in Kubernetes.

Recommended architecture

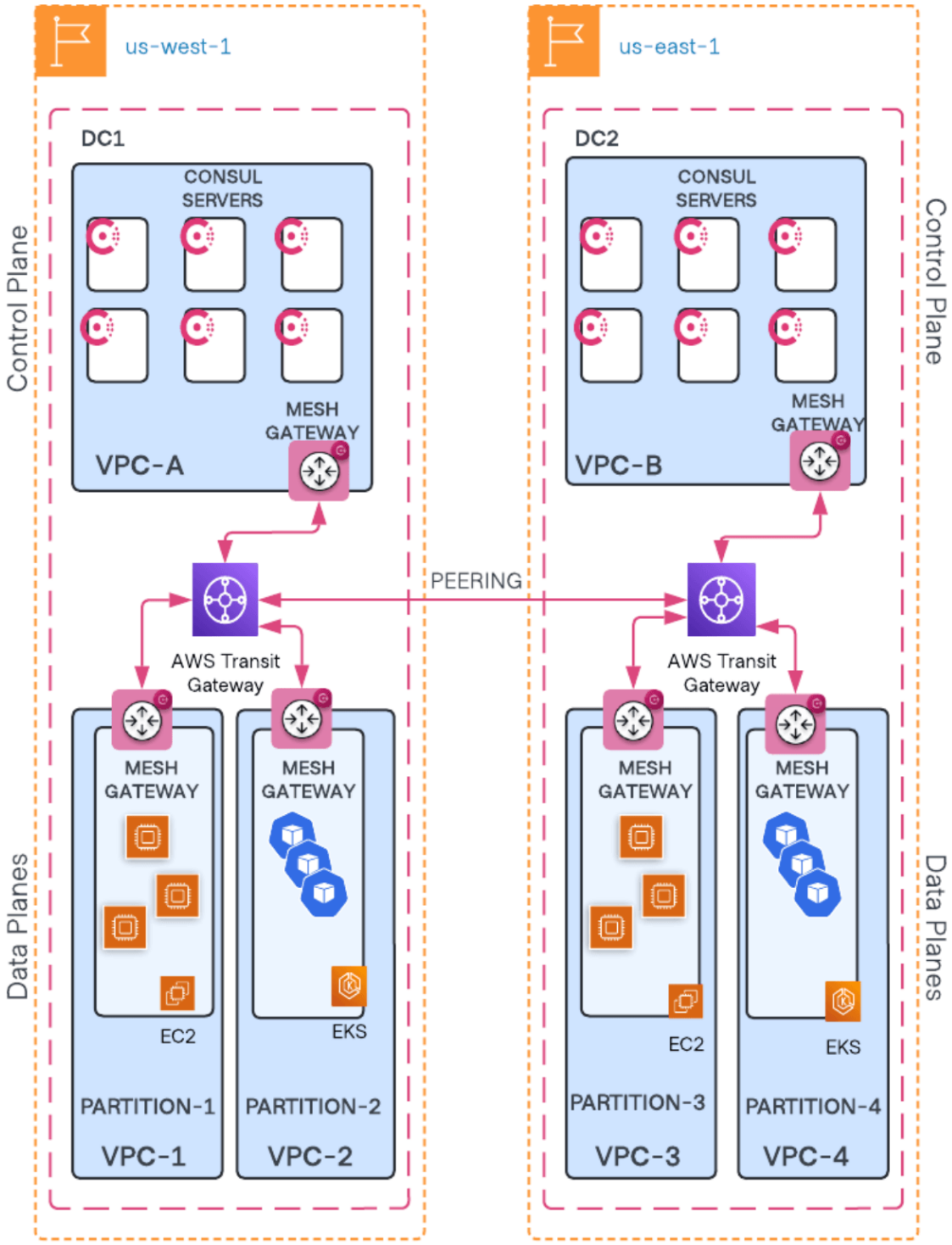


Figure 9: Consul multi-cluster recommended architecture

References

- Consul cluster peering overview
- Enabling peering control plane traffic (via mesh gateways)

11.5 Conclusion

Consul offers robust multi-tenancy features that facilitate the design and deployment of a self-service Consul platform, ensuring high resilience and scalability.

12 Support readiness

To ensure smooth communication and efficient support, make sure that these guidelines are followed:

- Each team member must have an account created through our support portal.
- Each team member that should be permitted to create support tickets on behalf of your company must be configured as an Authorized Technical Contact in the support portal. Please provide a list of applicable team members to your company's assigned Solutions Engineer so that they can configure this.
- Have your team familiarize themselves with our documentation on how to open a support ticket with HashiCorp Support and generate/upload a support bundle, whenever applicable.
- Be aware of the support plan level your company has purchased to manage expectations of response times. Also, understand the definitions for each severity level when opening a support ticket through our documentation.

13 Changelog

This page records notable changes to the Consul Administration Guide.

13.1 2026-08

- Restructured legacy Consul HVD content into the HVD 2.0 Consul Administration Guide as part of the Installation / Administration / User guide migration.

13.2 2026-01

- Update meta descriptions
- Updated credits and contributors

13.3 2025-06

- Use consistent guide names and descriptions

13.4 2025-05

- Fix kubernetes example config

13.5 2025-01

- Fix policies code
- Fix typos and documentation errors
- Updated documentation links
- Add new people process self service workflow diagram
- Add improved resolution graphic of Cloud Operating Model

13.6 2024-08

- Minor fixes to improve readability

13.7 2024-02

- GA release

14 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

List of key team members who contributed to the development of this guide. Hailing from various functional areas and representing diverse departments within HashiCorp, these experienced professionals have played pivotal roles in writing, editing, reviewing, and ensuring the completion of this material. Their collective efforts have been instrumental in shaping the final output.

- Abhishek Tiwari
- Adam Cavaliere
- Austin Workman
- Blake Covarrubias
- Çetin Ardal
- Chris Dunlap
- Cooper Melgreen
- David Cañadillas
- David Canadillas
- Gerald Dykeman
- Hari Sankaran
- Jan Repnak
- Josh Wolfer
- Kalen Arndt
- Kulpreet Singh
- Luke Kysow
- Mara Hammond
- Mark Lewis
- Pablo Diaz
- Prabhjit Singh
- Sean Doyle
- Srini Nagaraju
- Tu Nguyen